ALIBABA CLOUD

阿里云

EMAS

windvane 开发指南

产品版本：

文档版本：20211228

阿里云

法律声明

阿里云提醒您在阅读或使用本文档之前仔细阅读、充分理解本法律声明各条款的内容。如果您阅读或使用本文档，您的阅读或使用行为将被视为对本声明全部内容的认可。

- 您应当通过阿里云网站或阿里云提供的其他授权通道下载、获取本文档，且仅能用于自身的合法合规的业务活动。本文档的内容视为阿里云的保密信息，您应当严格遵守保密义务；未经阿里云事先书面同意，您不得向任何第三方披露本手册内容或提供给任何第三方使用。
- 未经阿里云事先书面许可，任何单位、公司或个人不得擅自摘抄、翻译、复制本文档内容的部分或全部，不得以任何方式或途径进行传播和宣传。
- 由于产品版本升级、调整或其他原因，本文档内容有可能变更。阿里云保留在没有任何通知或者提示下对本文档的内容进行修改的权利，并在阿里云授权通道中不时发布更新后的用户文档。您应当实时关注用户文档的版本变更并通过阿里云授权渠道下载、获取最新版的用户文档。
- 本文档仅作为用户使用阿里云产品及服务的参考性指引，阿里云以产品及服务的“现状”、“有缺陷”和“当前功能”的状态提供本文档。阿里云在现有技术的基础上尽最大努力提供相应的介绍及操作指引，但阿里云在此明确声明对本文档内容的准确性、完整性、适用性、可靠性等不作任何明示或暗示的保证。任何单位、公司或个人因为下载、使用或信赖本文档而发生任何差错或经济损失的，阿里云不承担任何法律责任。在任何情况下，阿里云均不对任何间接性、后果性、惩戒性、偶然性、特殊性或刑罚性的损害，包括用户使用或信赖本文档而遭受的利润损失，承担责任（即使阿里云已被告知该等损失的可能性）。
- 阿里云网站上所有内容，包括但不限于著作、产品、图片、档案、资讯、资料、网站架构、网站画面的安排、网页设计，均由阿里云和/或其关联公司依法拥有其知识产权，包括但不限于商标权、专利权、著作权、商业秘密等。非经阿里云和/或其关联公司书面同意，任何人不得擅自使用、修改、复制、公开传播、改变、散布、发行或公开发表阿里云网站、产品程序或内容。此外，未经阿里云事先书面同意，任何人不得为了任何营销、广告、促销或其他目的使用、公布或复制阿里云的名称（包括但不限于单独为或以组合形式包含“阿里云”、“Aliyun”、“万网”等阿里云和/或其关联公司品牌，上述品牌的附属标志及图案或任何类似公司名称、商号、商标、产品或服务名称、域名、图案标示、标志、标识或通过特定描述使第三方能够识别阿里云和/或其关联公司）。

- 如若发现本文档存在任何错误，请与阿里云取得联系。

通用约定

格式	说明	样例
危险	该类警示信息将导致系统重大变更甚至故障，或者导致人身伤害等结果。	危险 重置操作将丢失用户配置数据。
警告	该类警示信息可能会导致系统重大变更甚至故障，或者导致人身伤害等结果。	警告 重启操作将导致业务中断，恢复业务时间约十分钟。
注意	用于警示信息、补充说明等，是用户必须了解的内容。	注意 权重设置为0，该服务器不会再接受新请求。
说明	用于补充说明、最佳实践、窍门等，不是用户必须了解的内容。	说明 您也可以通过按Ctrl+A选中全部文件。
>	多级菜单递进。	单击设置> 网络> 设置网络类型。
粗体	表示按键、菜单、页面名称等UI元素。	在结果确认页面，单击确定。
`Courier字体`	命令或代码。	执行 `cd /d C:/window` 命令，进入Windows系统文件夹。
斜体	表示参数、变量。	`bae log list --instanceid` *Instance_ID*
[] 或者 [a|b]	表示可选项，至多选择一个。	`ipconfig [-all|-t]`
{} 或者 {a|b}	表示必选项，至多选择一个。	`switch {active|stand}`

目录

法律声明 2
通用约定 4
目录 .....
H5 相关技术 7
1. Android H5 容器指南 7
2. iOS H5 容器指南 18
3. JS API 指南 35
3.1. 接入 35
3.2. 规范与约束 37
3.3. 基础 Base 45
3.4. 窗口小部件 Widget 51
3.5. 声音播放 WVAudio 57
3.6. 相机 WVCamera 57
3.7. 通讯录 WVContacts 63
3.8. 文件 WVFile 65
3.9. 跨域 Cookie WVCookie 67
3.10. 地理位置 WVLocation 68
3.11. 网络检测 WVNetwork 70
3.12. 动作检测 WVMotion 71
3.13. 设备环境探测 WVNativeDetector 75
3.14. 蜂鸣 WVNotification 78
3.15. 截屏 WVScreen 79
3.16. 向 Native 发送事件 WVStandardEventCenter 81
3.17. 信息报告 WVReporter 83
3.18. 性能统计 WVWebPerformance 85
3.19. 应用信息 WVApplication 86
3.20. 图片保存 WVImage
3.21. 预加载包 WVPackageAppInfo 89
3.22. 其它特性 Other Features 90
4. 前端框架兼容性 92
5. 从其他框架迁移 101

H5 相关技术

1. Android H5 容器指南

初始化

添加依赖

```
compile ('com.emas.hybrid:emas-hybrid-android:1.0.3.0') { transitive true}
```

添加权限 activity 到清单文件

```
<activity android:name="android.taobao.windvane.runtimepermission.PermissionActivity">
</activity>
```

在 Application 中合适的位置进行 EmasHybrid 的初始化工作

```
// 自定制适配器（可选）
EmasHybrid.Config.Builder builder = new EmasHybrid.Config.Builder();
        builder.setNavigatorAdapter(new INavigatorAdapter() {
        //xxx
        }).setStorageAdapter(new IStorageAdapter() {
        //xxx
        });

EmasHybrid.getInstance()
    .setOpenLog(true)
    .setAppKey(mAppkey)
    .setAppSecret(mAppSecret)
    .setTtid(mChannelID)
    .setAppVersion(BuildConfig.VERSION_NAME)
    .setZcacheEnable(true)// 如果需要开启 zcache 预加载
```

```
.setZcacheUrl(mCacheURL)// zcache 预加载拉取地址前缀
.setConfig(builder.build())// 自定制适配器，可选
.init(application);
```

注：

- INavigatorAdapter Navigator 模块，系统提供了 pop/push 的实现，也可以自定义
- IStorageAdatper Storage 模块，系统提供了 setItem/getItem/removeItem 的默认实现，也可以自定义

撰写容器 Activity

为保证最大的集成灵活性，本 SDK 提供 Fragment 形式 H5 容器，可以借助所提供的 EmasHybridInstance 将 H5 容器 Fragment 渲染至您的 Activity 的任意合适的位置。

注册 activity

为保证内置的 Navigator 模块生效，请将您的 Activity 加入如下 IntentFilter 配置

```
<intent-filter>
  <action android:name="android.intent.action.VIEW"/>
  <category android:name="android.intent.category.BROWSABLE" />
  <category android:name="com.emas.android.intent.category.HYBRID" />
  <category android:name="android.intent.category.DEFAULT"/>

  <data android:scheme="http"/>
  <data android:scheme="https"/>
</intent-filter>
```

data 标签中的 scheme 取决于 Navigator 模块所需要跳转的 scheme，您可以自定义更多

将 Fragment 添加至 Activity

在您的布局文件中合适的位置添加一个 Framelayout，比如

```
<FrameLayout
    android:id="@+id/root"
    android:layout_width="match_parent"
```

```
    android:layout_height="wrap_content">

</FrameLayout>
```

在 Activity 中使用 EmasHybridInstance 渲染 Fragment

```
webViewFragment = EmasHybridWebViewHelper.getInstance().installWebView(
                this,
                getIntent().getData(),
                R.id.root,
                new WVWebViewClient(this) {
                    @Override
                    public void onPageFinished(WebView view, String url) {
                        super.onPageFinished(view, url);
                        Log.d("test", view.getTitle());
                    }
                }, null);
```

注意，渲染的 url 保存在 Intent 的 data 中

添加返回键和回调处理

```
@Override
public boolean onKeyDown(int keyCode, KeyEvent event) {
    if (keyCode == KeyEvent.KEYCODE_BACK && webViewFragment != null) {
        boolean result = EmasHybrid.getInstance().isUseUc() ? ((WVUCWebViewFragment)
webViewFragment).onBackPressed() : ((WVWebViewFragment) webViewFragment).onBackPressed();
        return result || super.onKeyDown(keyCode, event);
    }
    return super.onKeyDown(keyCode, event);
}
```

```
@Override
public void onActivityResult(int requestCode, int resultCode, Intent data) {
    if (EmasHybrid.getInstance().isUseUc()) {
        ((WVUCWebViewFragment) webViewFragment).onActivityResult(requestCode, resultCode,
data);
    } else {
        ((WVWebViewFragment) webViewFragment).onActivityResult(requestCode, resultCode,
data);
    }

}

@Override
protected void onPause() {
    if (EmasHybrid.getInstance().isUseUc()) {
        ((WVUCWebViewFragment) webViewFragment).onPause();
    } else {
        ((WVWebViewFragment) webViewFragment).onPause();
    }
    super.onPause();
}
@Override
protected void onResume() {
    if (EmasHybrid.getInstance().isUseUc()) {
        ((WVUCWebViewFragment) webViewFragment).onResume();
    } else {
        ((WVWebViewFragment) webViewFragment).onResume();
    }
```

```
    super.onResume();
}

@Override
protected void onDestroy() {
    if (EmasHybrid.getInstance().isUseUc()) {
        ((WVUCWebViewFragment) webViewFragment).onDestroy();
    } else {
        ((WVWebViewFragment) webViewFragment).onDestroy();
    }
    super.onDestroy();
}
```

使用 UC 内核

获取接入授权码

UCSDK 需要单独的接入授权码方可使用，请联系客户对应的解决方案工程师予以申请，需要提供应用包名和证书签名 SHA 串码

使用命令 keytool -list -v -keystore ~/.android/debug.keystore 生成所需证书的 SHA1 哈希值

UCSDK 接入

申请接入授权码在 EmasHybrid 初始化时通过创建 UCConfig 传入，示例如下

```
EmasHybrid.UCConfig.Builder builder = new EmasHybrid.UCConfig.Builder();
EmasHybrid.UCConfig ucConfig = builder
                .setEnable(true)
    .setCoreUnderWifiOnly(false)// 解除仅 4G 环境下载 UC 内核的限制
    .setUcSdkAppKeySec(new
String[]{"KeFcdGYMAZsavbV0VprwEy4nKq+zuMW0lcn43EWXUzx2fIDEtc5MZx+l8FykIQvJ8LLQV4XT2TBun9kg
UMRRdw==="})
                .build();
```

```
EmasHybrid.getInstance()
                ..
                .setUcConfig(ucConfig)
                .init(application);
```

使用 UCFragment

```
webViewFragment = EmasHybridWebViewHelper.getInstance().installUCWebView(
                this,
                getIntent().getData(),
                R.id.root,
                new WVUCWebViewClient(this) {
                    @Override
                    public void onPageFinished(com.uc.webview.export.WebView view, String
url) {
                        super.onPageFinished(view, url);
                    }
                }, null);


    @Override
    public boolean onKeyDown(int keyCode, KeyEvent event) {
        if (keyCode == KeyEvent.KEYCODE_BACK && webViewFragment != null) {
            boolean result = ((WVUCWebViewFragment) webViewFragment).onBackPressed();
            return result || super.onKeyDown(keyCode, event);
        }
        return super.onKeyDown(keyCode, event);
    }
```

其他步骤都与集成系统 webview 相同

确认内核类型

Useragent 信息中，如果包含 UCBrowser 字段，则是 UC 内核(如果还包含了 UCBS 字段，则是 U4 内核，否则为 U3 内核)，如果以上字段都没有，是系统原生的内核。

可以扫下面的页面，会展示出内核的类型：

数据监控

EMAS-Hybrid 提供了默认的、全面的数据监控实现，默认开启，可以在初始化的时候手动关闭

```
EmasHybrid.getInstance()
    .setOpenMonitor(true)//默认为开启
    .init(application);
```

自定义自己的实现

在保留默认性能监控逻辑的基础上，客户可以自己追加实现，可选的继承类如下：

类名	作用
WVMonitorImpl.java	性能和错误监控

这里以客户自己追加实现首屏加载实现举例，首先继承自 WVMonitorImpl，并实现其中两个方法，注意保证每个方法的 super 方法的调用

```
public class CustomPerformanceMonitor extends WVMonitorImpl {

    private static long start = 0;

    @Override
```

```
    public void didPageStartLoadAtTime(String url, long time) {
        super.didPageStartLoadAtTime(url, time);
        start = time;

    }

    @Override
    public void didPageFinishLoadAtTime(String url, long time) {
        super.didPageFinishLoadAtTime(url, time);
        long elapse = time - start;
        Log.d("test", ">>> " + elapse);
    }
}
```

在 EmasHybrid 初始化后注册到 EmasHybridMonitor

```
EmasHybrid.getInstance().xxx.init(application);
EmasHybridMonitor.registerPerformanceMonitor(new CustomPerformanceMonitor());
```

使用 JS Bridge

EMASHybrid 容器接入后，您即获得了相当多的实用内置的 JS API，相关的 API 参考文档如下：

https://help.aliyun.com/document_detail/102685.html

如果您有自己的原生实现需要通过 EMASHybrid 的 JS Bridge 暴露出能力供前端调用，也是很容易的。这里以如何扩展一个 Toast 为例，首先加入你的原生插件实现：

```
public class CustomToastPlugin extends WVApiPlugin {
    @Override
    protected boolean execute(String action, String params, WVCallBackContext
wvCallBackContext) {
        JSONObject jsonObject = null;
        try {
```

```
            jsonObject = new JSONObject(params);
            String content = jsonObject.optString("content");
            if ("long".equals(action)) {
                Toast.makeText(mContext, content, Toast.LENGTH_LONG).show();
            } else if ("short".equals(action)) {
                Toast.makeText(mContext, content, Toast.LENGTH_SHORT).show();
            }
        } catch (JSONException e) {
            e.printStackTrace();
        }

        return true;
    }
}
```

再在 EMASHybrid 初始化后调用插件管理器注册这个插件：

```
EmasHybridApi.registerPlugin("Toast", CustomToastPlugin.class);
```

这样就成功的注册了一个叫做 Toast 的插件，它含有两个函数 short 和 long，需要前端传递叫做 content 的参数，即 Toast 需要显示的内容，前端可以按照下列方式调用这个 JS API：

```
let params = {
 content: "This is a short Toast!"
}
window.WindVane.call('Toast', 'short', params, function(e) {
 }, function(e) {
 alert('failure' + JSON.stringify(e));
});
```

当然，也可以通过 lib-emas 来预先注册这个 JS API，通过调用 emas.add 函数：

```
emas.add("Toast.long")
emas.add("Toast.short")
```

随后就可以调用这两个函数：

```
emas.Toast.long({content: 'Hello'})
emas.Toast.short({content: 'Hello'})
```

标准事件机制

提供了 Native 和 H5 之间互发事件的标准通讯接口，便于各业务场景下的基于统一的事件模型来通讯

Native 向 H5 发送事件

EMASHybrid 提供了 WVStandardEventCenter 类，供 Native 向 H5 发送事件：

参数中，webView 是要发送事件的目标 WebView，eventName 是事件的名称（字符串），eventData 是事件的数据（iOS 是 NSDictionary *，Android 是一个 JSON Object 格式的 String）。

```
// 向指定 WebView 发送事件。
WVStandardEventCenter.postNotificationToJS(webView, eventName, eventData);
// 向所有 WebView 发送事件，请谨慎使用。
WVStandardEventCenter.postNotificationToJS(eventName, eventData);
```

随后前端中使用 W3C 的标准形式来接收事件：

```
document.addEventListener('eventName', function(data) {
 // 这里要注意，Native 传递过来的事件参数是在 data 的 param 属性中。
 alert(data.param);
}, false);
```

H5 向 Native 发送事件

EMAS Hybrid 提供了发送事件的 JS API 供前端调用：

```
var params = {
 event: 'eventName',
```

```
  param: {
   // 事件要传递的数据。
  }
 };
 window.WindVane.call('WVStandardEventCenter','postNotificationToNative', params,
 function(e) {
  alert('success');
 }, function(e) {
  alert('failure: ' + JSON.stringify(e));
 });
```

或者使用 lib-emas 的调用方式

```
 emas.WVStandardEventCenter.postNotificationToNative({...})
```

原生若需要接受事件，需要注册 WVEventListener 监听器：

```
 // 首先，需要自己实现监听用的 Listener。
 public class JsEventListener implements WVEventListener {
  @Override
  public WVEventResult onEvent(int id, WVEventContext ctx, Object... obj) {
   // 所有事件均派发到这里，WVEventId.H5TONATIVE_EVENT 表示 H5 发送过来的事件。
   if (id == WVEventId.H5TONATIVE_EVENT) {
    if (obj[0] instanceof String) {
     String params = (String)obj[0];
     // 这里的 params 是包含 event 和 param 的 JSON String，需要自行反序列化。
    }
   }
   return null;
  }
```

```
}
// 然后，注册监听器。
WVEventService.getInstance().addEventListener(new JsEventListener());
```

2. iOS H5 容器指南

接入

- **整包获取(推荐)**

整包的 WindVane 包含了所有功能

```
source 'git@gitlab-ce.emas-poc.com:EMAS-iOS/emas-specs.git'

pod 'WindVane' ,'1.0.0'
```

- **分包获取**

```
source 'git@gitlab-ce.emas-poc.com:EMAS-iOS/emas-specs.git'

pod 'WindVaneCore','1.0.0'
pod 'WindVaneBridge','1.0.0'
pod 'WindVaneBasic','1.0.0'
pod 'WindVaneAPI','1.0.0'
pod 'WindVaneMonitor','1.0.0'
pod 'WindVaneEMASExtension','1.0.0'
```

- **功能说明**

WindVaneCore

WindVane 核心库，定义了模块间的接口和约束，以及公共工具类，所有模块都必须依赖它。

WindVaneBridge

WindVane JSBridge 库，提供了 JSBridge 的功能，可以在自己的 WebView 中或者直接脱离 WebView 使用。

WindVaneBasic

WindVane 基础 WebView 库，提供了 WebView 相关功能优化和扩展，包括 WebView、WKWebView、ViewController、StandardEventModal 模块。

WindVaneAPI

WindVane 自身提供的一些基础 JSBridge API。 需要添加系统库:CoreBluetooth.framework、AddressBookUI.framework、AddressBook.framework、MessageUI.framework、Messages.framework、ContactsUI.framework、Contacts.framework、UserNotifications.framework

WindVaneMonitor

WindVane 埋点支持库，提供了埋点的 *UT* 通道，允许将 WindVane 的埋点情况上传到 EMAS 平台，包括 H5 性能埋点，错误埋点，JSBridge 埋点和预加载埋点等。

WindVaneEMASExtension

EMAS 扩展库，主要有 url 拦截使用预加载配置和上报 crash，预加载可以让前端在 EMAS 平台将 html、js、css 等资源打包成一个 zip 包，客户端可以采用启动后下载或者直接预置的方式将 zip 包解压到本地，使得用户在浏览网页的时候不需要再去下载资源，提升用户体验。

```
pod 'ZCache', '10.0.3'
```

初始化和配置

如下配置好对应 EMAS 平台的配置

```
@interface EMASWindVaneConfig : NSObject

+ (void)setUpWindVanePlugin;

@end

@implementation EMASWindVaneConfig

+ (void)setUpWindVanePlugin {
    // 设置 APPKey，如果使用了安全黑匣子，就会使用安全黑匣子的 key
    //    [WVUserConfig setAppKey:@"4272" secrect:@"0ebbcccfee18d7ad1aebc5b135ffa906"];
    [WVUserConfig setAppKey:[[EMASService shareInstance] appkey]];
    // 设置是否使用安全黑匣子
```

```
 [WVUserConfig useSafeSecert:NO];
    // 设置环境
    // [WVUserConfig setEnvironment:WVEnvironmentDaily];
    [WVUserConfig setEnvironment:WVEnvironmentRelease];
    // 设置 TTID
    //[WVUserConfig setTTid:@"windvane@demo2"];
    // 设置 UA
    [WVUserConfig setAppUA:[NSString stringWithFormat:@"TBIOS"]];
    // 设置 App 名称，会在 UserAgent 中带上，请务必正确设置。
    [WVUserConfig setAppName:@"EMASDemo"];
    // 设置 app 版本
    NSDictionary *infoDictionary = [[NSBundle mainBundle] infoDictionary];
    [WVUserConfig setAppVersion: [infoDictionary
objectForKey:@"CFBundleShortVersionString"]];
    // WKWebView 支持 NSURLProtocol
    [WVURLProtocolService setSupportWKURLProtocol:YES];

#ifdef DEBUG
    [WVBasicUserConfig setDebugMode:YES];
    // 打开 WindVane 的 Log
    [WVBasicUserConfig openWindVaneLog];
    [WVUserConfig setLogLevel:WVLogLevelVerbose];
    [WVBasic setJSLogLevel:WVLogLevelVerbose];
#endif

    // 初始化 WindVane 各模块
    //[WVTBExtension setup];
    [WVBasic setup];
    [WVAPI setup];
```

```
    [WVMonitor startMonitoring];
    [EMASWVExtensionConfig setup];
}

@end
```

在应用程序初始化时

```
- (BOOL)application:(UIApplication *)application
didFinishLaunchingWithOptions:(NSDictionary *)launchOptions {
      [self initZCacheConfig];
      [EMASWindVaneConfig setUpWindVanePlugin];
}
```

另外:ZCache 初始化参考基础 SDK 初始化部分

常见错误

如果引入 WindVane 时报如下的错误，那么可以在项目的 *Build Phrase* 中的 *Link Binary With Libraries* 中添加 WebKit，并选择 Status 为 Optional。

[MISSING IMAGE: error message | left | 618x118, error message | left | 618x118]
[MISSING IMAGE: WebKit framework | left | 479x80, WebKit framework | left | 479x80]

使用 iOS WebView

WindVane 在 iOS 平台提供了 WVWebView（基于 UIWebView） 和 WVWKWebView（基于 WKWebView） 的支持，并为两种 WebView 做了功能和接口的兼容。因此请总是使用 UIView * 作为 WebView 的类型，可以兼容 WVWebView 和 WVWKWebView 的主要功能。

另外，请总是使用 #import <WindVane/WindVane.h> 来引入 WindVane 的头文件，无需区分 WindVane 整包或者分包。业务方在升级 WindVane 版本时，请注意检查并修改代码的 Warning，尽早修改已废弃的方法。

使用 WindVane 提供的 ViewController

首先您可以选择使用 WindVane 提供的含有 WebView 的 WVUIWebViewController，它封装了常用的 WebView 接口，提供了 WVWebView 和 WVWKWebView 切换、状态栏背景兼容、工具栏、错误页、加载框和横屏支持等功能。

UI 无关的基础功能请参考 WVViewController.h，UI 相关的功能请参考 WVUIWebViewController.h。

要在 WVUIWebViewController 中切换到 WVWKWebView，可以设置 useWKWebView 属性：

- WVUseWKWebViewNever 表示总是使用 WVWebView，是 useWKWebView 属性的默认值。
- WVUseWKWebViewAlways 表示总是使用 WVWKWebView。
- WVUseWKWebViewCustom 表示在创建 WebView 时，根据 decideIsUseWKWebView 方法的返回值决定是使用 WVWKWebView，还是 WVWebView。由于必须在初始化 WebView 之前就决定是否切换到 WVWKWebView，因此 WVUIWebViewController 默认会检查 loadUrl 中是否包含 _wvUseWKWebView=YES 参数，包含的话就切换到 WVWKWebView。

```
#import <WindVane/WindVane.h>

WVUIWebViewController * controller = [[WVUIWebViewController alloc] init];

controller.loadUrl = @"http://m.taobao.com";

// 开启 JSBridge，如果需要使用 JSBridge 必须调用。

controller.openLocalService = YES;

// [iOS 7 适配] 隐藏导航栏时，为 Status Bar 添加白色背景颜色，按需要设置。

[controller supportiOS7WithoutStatusBar];

// 禁用 WebView 的长按事件，按需要设置。

// controller.openWebKitLongPress = NO;

[self.navigationController pushViewController:controller animated:YES];
```

默认的 WebAppInterface 实现

WindVane iOS 中内置了常用 WebAppInterface 接口的默认实现，具体包含：

- setNaviBarHidden：隐藏或显示导航栏，隐藏导航栏时总是同时隐藏状态栏
- setCustomPageTitle：设置导航栏标题
- setNaviBarStyle：设置导航栏前景色和背景色
- setStatusBarStyle：设置状态栏前景色
- setNaviBarRightItem：设置导航栏右侧按钮
- clearNaviBarRightItem：清除导航栏右侧按钮
- forbidPanGesture：禁用手势返回
- pop：回退当前 ViewController，优先回退历史记录

  对应的 Native 接口，全部列出在了 WVWebViewControllerUIProtocol.h 中，根据需要，可以自行重写 WVUIWebViewController 提供的默认实现（注意在 ViewController appear/disappear 时要正确重置样式）。也可以只重写 WVUIWebViewController+CustomizeUI.h 中列出的更新接口，由 WindVane 在合适的时机完成样式的重置。 WVUIWebViewController.allowsControlNavigationBar 的默认值为 NO，表示禁用以上 WebAppInterface 接口。业务方需要自行将该属性设置为 YES，以启用该功能。

使用 WebView

如果您不希望直接使用 WVUIWebViewController，可以直接创建 WVWebView 或 WVWKWebView，但建议**使用 UIView<WVWebViewProtocol> * 作为 WebView 的类型**，便于动态切换 WebView 类型。特别是**请总是使用 evaluateJavaScript:completionHandler: 异步接口来执行 JS**。

WebView 本身提供了 JSBridge、下拉刷新、上拉更多、长按保存图片、加载进度和页面性能监测等功能，具体请参考 WVWebViewProtocol.h。

WVWebViewProtocol 抹平了绝大部分 UIWebView 和 WKWebView 的差异，目前仍有以下细节区别： 1. allowsInlineMediaPlayback 和 mediaPlaybackRequiresUserAction 在 WKWebView 初始化后再设置无法生效，必须在 initWithFrame:configuration: 的 configuration 中设置。 2. evaluateJavaScript:completionHandler: 的返回值序列化方式略有不同。如果方法的返回值不是字符串，UIWebView 会转换为 NSString *，而 WKWebView 会序列化为与 JavaScript 类型对应的 Native 类型。建议总是使用字符串作为 JavaScrip 返回值。使用时**必须正确设置 WebView 的 sourceViewController 属性**，并在 ViewController 的 viewWillAppear 时调用 bindingWebViewService 方法，在 viewWillDisappear 时调用 releaseWebViewService 方法。

```
// 创建 WebView 时调用：
_webView.sourceViewController = self;
// WebView 的 JSBridge 功能默认是关闭的，需要主动开启。
_webView.openLocalService = YES;

(void)viewWillDisappear:(BOOL)animated {
[super viewWillDisappear:animated];
[_webView releaseWebViewService];
}
(void)viewWillAppear:(BOOL)animated {
[super viewWillAppear:animated];
[_webView bindingWebViewService];
}
(void)dealloc {
// dealloc 中要正确释放 WebView。
_webView = nil;
}
```

使用 WKWKWebView

建议在 iOS9 或更高版本再使用 WKWebView，iOS8 中存在较多 bug。

WVWKWebView 使用的是 iOS 8.0 新加入的 WKWebView，使用 WebKit 内核进行渲染，具有更高的渲染效率和 JS 执行效率，也支持更多的 HTML5 特性。有需要的 Native 业务方可以根据自己的需要，使用 WVWKWebView 来展示页面。

WindVane iOS 版本中，全面优化了对 WVWKWebView 的支持，通过 [WVURLProtocolService setSupportWKURLProtocol:YES] 主动开启 NSURLProtocol 的拦截功能后，可以正常使用预加载功能，持久化 Cookie（设置有 expires 或 max-age）也可以正常同步，但 POST 请求会丢失 body，非持久化 Cookie 仍是不能同步的。目前仅有 XMLHTTPRequest 同步 POST 请求 Blob 和 FormData 不支持。

监听页面信息

WindVane 提供了页面的基础信息。通过 KVO 监听 WebView title 属性和 estimatedProgress 属性的变化，就可以及时获取页面标题和加载进度的改变，然后客户端自行决定如何显示在界面中。

```
// 添加 KVO 监听，务必在 WebView 销毁前移除监听。
[_webView addObserver:self forKeyPath:@"title" options:NSKeyValueObservingOptionNew
context:nil];
[_webView addObserver:self forKeyPath:@"estimatedProgress"
options:NSKeyValueObservingOptionNew context:nil];
// 处理 KVO 监听。
- (void)observeValueForKeyPath:(NSString *)keyPath ofObject:(id)object
change:(NSDictionary *)change context:(void *)context {
 [super observeValueForKeyPath:keyPath ofObject:object change:change context:context];
 if ([keyPath isEqual:@"title"]) {
  // change[NSKeyValueChangeNewKey] 就是新的页面标题。
 } else if ([keyPath isEqual:@"estimatedProgress"]) {
  // [change[NSKeyValueChangeNewKey] doubleValue] 就是新的加载进度，范围是 0.0~1.0。
  // 注意加载进度可能在非主线程触发，如果有 UI 操作注意切换线程。
 }
}
- (void)dealloc {
 // 移除 KVO 监听，这里只是一个示例，根据实际情况不一定要在 dealloc 中做。
 [_webView removeObserver:self forKeyPath:@"title"];
```

```
  [_webView removeObserver:self forKeyPath:@"estimatedProgress"];
}
```

使用 JSBridge

简介

WindVane JSBridge 提供了 JS 与 Native 之间的通信链路，基于这个通信基础，Native 可以暴露出一些服务 API 提供给 JS 调用，并通过方法返回值或事件将数据返回给 JS。我们提供了相当多的实用内置的 JS API，相关的 API 参考文档如下： https://help.aliyun.com/document_detail/102685.html

JS 端会使用 ClassName.HandlerName 来调用 JSBridge，WindVane 会根据 ClassName 与 HandlerName，来找到对应的 Native 方法。

JSBridge 的 Native 方法提供方，要像提供普通方法一样提供 JSBridge，**要求必须满足以下基本约束：**

1. 方法调用后，必须返回，任何情况下都要调用 callback 以将结果返回给 JS；可以根据需要返回不同的数据。
2. 方法只能有一个返回值，也就是说 **callback 只允许调用一次，如果有多个数据需要返回给 H5，Android 请使用__标准事件机制**，iOS 请使用 __JSBridge context 的 dispatchEvent 方法。
3. 注意做好资源的释放工作，尤其 iOS 不允许对 View 或 ViewController 有强引用。

注册 JSBridge

JSBridge 需要注册给 WindVane，才能够让 JS 正确的调用到对应的 Native 方法。我们提供了两种注册方案——动态注册和静态注册：

- 动态注册无需主动注册给 WindVane，使用简单方便。但是要求类名与 JSBridge 调用时的 ClassName 相同，且目前仅限 WindVane iOS。
- 静态注册需要调用 WindVane 的注册方法，使用略微麻烦。不要求特定的类名，会更加灵活。

总得来说，只要 JSBridge 调用的 ClassName 并没有被占用，那么总是建议使用动态注册的方式。

iOS 动态注册 JSBridge

WindVane 提供了新版本的用法，在 WVBridgeCallbackContext 中统一提供了 JSBridge 需要的所有功能，而且为 Weex 等非 WebView 场景也提供了支持。详情请参见下面的新接口介绍。

假设需要实现两个 JSBridge：MyJSBridge.firstAPI 和 MyJSBridge.secondAPI，那么 JSBridge 的 ClassName 是 MyJSBridge，HandlerName 分别是 firstAPI 和 secondAPI。

1. 创建一个 **MyJSBridge** 类（动态注册要求与 JSBridge 调用时的 ClassName 相同），需要确保不会有同名类。
2. 令 MyJSBridge 类继承自 WVDynamicHandler（引入头文件 WVDynamicHandler.h）。

3. 实现 firstAPI 和 secondAPI 方法，要求方法的签名必须为-/+ (void)handlerName:(NSDictionary *)params withWVBridgeContext:(id<WVBridgeCallbackContext>)context 其中：
   - 方法可以是静态方法，也可以是动态方法。实例方法会直接调用，实例方法会先创建一个实例，然后在实例上调用。
   - 方法名与 JSBridge 调用的 HandlerName 相同，这里为 firstAPI 和 secondAPI。
   - 第一个参数为 NSDictionary *，是 JS 传入的参数对象。
   - 第二个参数为 id<WVBridgeCallbackContext>，是 JSBridge 的调用上下文。
4. 在方法中实现 JSBridge 的逻辑，使用 [context callbackSuccess:RESULT]/[context callbackFailure:RET withResult:RESULT] 来输出执行成功/失败的结果返回给 JSBridge 调用方。请保证__任何情况下都会调用、且只会调用一次［context callbackXXX］系列方法__。

注意：

1. JSBridge 总是在主线程调用，如果有耗时操作，请务必自行切换线程。
2. 可能你提供的服务带有多个阶段输出性，请使用 [context dispatchEvent:eventName withParam:param] 将结果通过事件的方式输出给 JSBridge 调用方。
3. 当你的 JSBridge 逻辑执行完毕，可以主动调用 [context releaseHandler:self] 方法来主动释放内存，否则就只能等到页面销毁的时候才会释放。注意在 dealloc 中调用是没有意义的，因此此时对象已经被释放了。

```
#import <Foundation/Foundation.h>

#import <WindVane/WindVane.h>

@interface MyJSBridge : WVDynamicHandler

@end

@implementation MyJSBridge

// 这是一个静态方法，直接调用。

+ (void)firstAPI:(NSDictionary *)params

withWVBridgeContext:(id<WVBridgeCallbackContext>)context {

 // Do something

 [context callbackSuccess:nil];

}

// 这时一个实例方法，会创建一个新实例，然后再调用。

- (void)secondAPI:(NSDictionary *)params

withWVBridgeContext:(id<WVBridgeCallbackContext>)context {

 // Do something
```

```
  [context callbackSuccess:nil];
}
@end
```

动态注册的 JSBridge，要求类名不能重复。若希望扩展现有 ClassName，可以使用 JSBridge 别名。

自定义生命周期

如果实现的是实例方法，默认情况下，每次 JSBridge 调用都会创建一个新实例；该实例会被强引用已保证不会被错误释放，直到 WebView 被销毁时才会释放。 可以实现以下方法，实现实例生命周期的自定义：

```
@implementation MyJSBridge
+ (WVBridgeScope)instanceScope {
  return WVBridgeScopeInvocation;
}
@end
```

WindVane 支持四种生命周期：每次调用一个实例（WVBridgeScopeInvocation），每个页面一个实例（WVBridgeScopePage），每个 View 一个实例（WVBridgeScopeView）和全局唯一实例（WVBridgeScopeStatic）。

对于 ScopeInvocation 和 ScopePage，实例的引用会持续到页面被切换（例如 WebView 跳转新页面、回退历史记录等）；对于 ScopeView，实例的引用会持续到 View 被销毁。因此在确保当前实例可以被销毁时，建议主动调用 [context releaseHandler:self] 立刻释放当前实例，以节约内存。ScopeStatic 的实例，会一直存在，但也支持通过 [context releaseHandler:self] 主动释放。

与 View 或 ViewController 交互

JSBridge 中允许与 View（包括 WebView，以及 Weex 的 View 等） 或 ViewController 进行交互，已实现某些特殊逻辑。 或者检查 ViewController 是否具有特定的类型，已实现只允许在某类 ViewController 中调用的私有 JSBridge。

不允许对 View 或 ViewController 持有强引用，防止出现循环引用导致内存泄露。context 已提供 view 属性和 viewController 供实例方法使用，在 WebView 环境下提供了 webViewEnv 属性，Weex 场景下提供了 weexEnv 属性。这些属性会自动设置和更新，可以直接使用。

发送事件

在 JSBridge 中向容器发送事件，请使用 [context dispatchEvent:withParam:]，会对当前环境（WebView/Weex）做适配。

事件机制中提供的方法只能够支持 WebView 环境。

复杂的 JSBridge 生命周期管理

对于一些持续性的 JSBridge，如音频播放、动作感应，需要长时间的发送事件。但是在用户跳转到其它 Native 页面、跳转到其它 H5 页面时，可能需要暂停音频播放和动作感应。 WindVane 提供了以下生命周期回调方法，可以在 JSBridge 中实现：

```
@implementation MyJSBridge
/**
 * 重置处理器 - 该方法将会在加载新页面时调用，可以做一些清理工作。
 *
 * @param context WVBridge 被重置时的上下文。
 * @param request 要加载的新页面。
 */
- (void)resetWithContext:(id<WVBridgeContext>)context withNextRequest:(NSURLRequest
*)request {
}
/**
 * 暂停处理器 - 该方法将会在 UIViewController viewWillDisappear 时调用，用于降低 WVBridge
API 的性能消耗，例如暂停播放音乐和持续性监听器等。
 * 不会对 WVBridgeScopeStatic 作用域的 WVBridge 调用。
 *
 * @param context WVBridge 被暂停时的上下文。
 */
- (void)pauseWithContext:(id<WVBridgeContext>)context {
}
/**
 * 恢复处理器 - 该方法将会在 UIViewController viewWillAppear 时调用，用于恢复 WVBridge API
的性能，例如恢复播放音乐和持续性监听器等。
 * 不会对 WVBridgeScopeStatic 作用域的 WVBridge 调用。
 *
 * @param context WVBridge 被恢复时的上下文。
```

```
 */
- (void)resumeWithContext:(id<WVBridgeContext>)context {
}
@end
```

如果 JSBridge 需要在切换到新页面后自动清理，需要实现 resetWithContext:withNextRequest:；如果需要在当前 UIViewController 被其它 Native 界面覆盖时暂停和恢复，需要实现 pauseWithContext: 和 resumeWithContex:。一般建议与 WVBridgeScopeView 同时使用。

如果需要在应用切换前后台时做特殊工作，可以自行注册 UIApplicationDidBecomeActiveNotification 和 UIApplicationWillResignActiveNotification 这两个通知。

新接口介绍

在新版本 JSBridge 接口，将所有功能收拢到了统一的 id<WVBridgeCallbackContext> 中，一切功能都可以通过 context 来实现，并且对 Weex 等非 WebView 环境提供了支持。具体包括：

1. 获取环境信息，基本信息有来源 URL referrer，view 和 viewController。以及当前的环境 env，在 WebView 环境中可以拿到当前 UIView<WVWebViewBasicProtocol> *，在 Weex 环境中可以拿到当前 WeexSDKInstance *。
2. 发送事件，提供了 dispatchEvent:withParam: 方法。
3. 释放实例，提供了 releaseHandler: 方法。
4. 回调，提供了 callbackSuccess: 和 callbackFailure:withResult: 方法，以及一系列失败回调的快捷方法。
5. 所有 JSBridge 的方法参数全部统一为 NSDictioanry * params 和 id<WVBridgeCallbackContext> context，无论是动态注册，还是各类静态注册的方法。

支持懒加载的动态库

一些客户端为了提高应用打开速度，对动态库做了懒加载处理，这样可以将动态库的加载延迟到使用时，避免影响应用启动耗时。

如果动态库中提供了动态注册 JSBridge，那么在动态库被加载之前，是找不到对应的类的，也无法正常调用。因此我们懒加载的动态库做了特别的支持，允许动态库通过 plist 配置动态 JSBridge 的类名，以便 WindVane 在需要使用 JSBridge 时主动将动态库加载进来。

解决方案是在动态库的 framework 根目录，提供一个命名为 XXX_bundle.plist 的文件，其中的 XXX 部分可以任意命名，甚至同一个 framework 包含多个符合此规则的 plist 也可以。在 plist 中添加名为 windvane_bridge_list 的数组，并在数组中添加需要配置的动态 JSBridge 类名，多个 plist 的配置会自动合并。下图是一个简单的示例：

[MISSING IMAGE: Dynamic Framework Config | left | 600x374, Dynamic Framework Config | left | 600x374]

请确保动态库全部位于客户端的 Frameworks 目录，如果不同动态库之间的映射关系存在覆盖，会在动态库提供的 JSBridge 被首次使用时打出日志 'A' 注册的 WVBridge 'XXX' 被 'B' 覆盖了，在 WindVane 的 DEBUG 模式下还会弹出 Alert 提示。

iOS 静态注册 JSBridge

WindVane iOS 允许将 JSBridge 注册到全局，也可以注册到特定 WebView。前者可以在任意 WebView 中调用，而后者只能在注册的 WebView 中使用，适合需要较严格的权限控制的 JSBridge。 但是注册到全局的 JSBridge，也可以通过对 ViewController 的类型或属性进行校验，已达到部分权限的控制，因此__如无特别的必要，请总是将 JSBridge 注册到全局__，会有更少的内存消耗。

将 JSBridge 注册到全局

这样注册的 JSBridge，在任意 WebView 或 Weex 页面中都可以使用 className 和 handlerName 调用。

```
#import <WindVane/WindVane.h>
// JSBridge 调用的 Block。
WVBridgeHandler handler = ^(NSDictionary * params, id<WVBridgeCallbackContext> context) {
 [context callbackSuccess:nil];
};
// 注册 JSBridge 到全局，特别注意这里的 JSBridge 名称合并写作 @"className.handlerName" 格式。
WVBridgeRegisterHandler(@"className.handlerName", handler);
```

将 JSBridge 注册到特定 WebView

这样注册的 JSBridge，只能在注册到的 WebView 中使用 className 和 handlerName 调用。

```
#import <WindVane/WindVane.h>
// JSBridge 调用的 Block。
WVBridgeHandler handler = ^(NSDictionary * params, id<WVBridgeCallbackContext> context) {
 [context callbackSuccess:nil];
};
// 注册 JSBridge 到指定 WebView，特别注意这里的 JSBridge 名称合并写作
@"className.handlerName" 格式。
[webview registerHandler:@"className.handlerName" withBlock:handler];
```

```
// 注册 JSBridge 到指定 ViewController 中的 WebView，特别注意这里的 JSBridge 名称合并写作 @"className.handlerName" 格式。
[wvViewController registerHandler:@"className.handlerName" withBlock:handler];
```

静态注册的 JSBridge，不支持复杂的生命周期管理。如果需要实现复杂的 JSBridge，请动态注册 JSBridge。

静态注册的 JSBridge，若 className 和 handlerName 都相同，那么后注册的会覆盖先注册的。同时也会覆盖同名的动态注册方法。

JSBridge 别名

WindVane 提供了 JSBridge 的别名服务，可以用来对现有 ClassName 的 JSBridge 进行扩展或覆盖，也可以对 JSBridge 的升级、更新、改名和冲突提供解决方案。

允许将原始的 JSBridge className.handlerName，映射到新的别名 aliasClassName.aliasHandlerName，用户即可以使用旧的 className.handlerName 来访问，也可以使用新的 aliasClassName.aliasHandlerName 来访问。

WindVane 的别名服务，是首先将别名解析为原始名称，然后使用原始名称来查找 JSBridge；**不要将别名指向其它别名，可能导致无法正确找到 JSBridge**。

iOS 注册全局别名

全局别名必须与全局 JSBridge 配合使用。

```
#import <WindVane/WindVane.h>
// 注册 JSBridge 别名到全局。
NSDictionary * alias = @{
 @"类别名.方法别名": @"类名.方法名",
 @"aliasClassName.aliasHandlerName": @"className.handlerName"
};
WVBridgeRegisterAlias(alias);
```

JSBridge 全局鉴权

WindVane 提供了 JSBridge 的全局鉴权接口，供实现 App 级别的 JSBridge API 鉴权。该鉴权层切入到 JSBridge 的调用逻辑中，可以由客户端统一自定义鉴权逻辑。

对于单个 JSBridge 的鉴权，请在该 JSBridge 的实现中自行实现。

```
#import <WindVane/WindVane.h>
// 实现 JSBridge 鉴权方法。
@interface MyJSBridgeCheckerHandler : NSObject <WVBridgeCheckerProtocol>
@end
@implementation MyJSBridgeCheckerHandler
/**
 * 检查指定 WVBridge 是否具有执行权限。
 * 总是在后台线程调用。
 *
 * @param apiName WVBridge 的名称，格式为 "类名.方法名"。
 * @param params  WVBridge 的调用参数。
 * @param context WVBridge 的执行上下文。
 *
 * @return 权限检查结果。
 */
- (WVBridgePermission *)checkPermission:(NSString *)apiName withParams:(NSDictionary
*)params withContext:(id<WVBridgeContext>)context {
 return [WVBridgePermission permissionNotSure];
}
@end
// 注册鉴权处理器。
[WVBridgeChecker registerChecker:[[MyJSBridgeCheckerHandler alloc] init]];
```

在 Native 中使用 JSBridge

WindVane iOS 中，支持在 Native 中调用 JSBridge。现在提供有两种用法：

简单用法

这里适合一些简单场景下，Native 调　用与 UI 无关的 JSBridge。直接调用 [WVBridge callHandler:withParams:withCallback:]方法，传入以下参数：

- name: 要执行的 JSBridge 名称，格式为 @"类名.方法名"

- params：要执行的 JSBridge 参数，与 JS 使用方式保持一致（对象用 NSDictionary 代替，数组用 NSArray 代替）
- callback：此次 JSBridge 调用的回调，**可能在任意线程调用**

复杂用法

这里适合 Native 调用全功能的 JSBridge，可以用于在自己的容器中完整接入 JSBridge。

1. 实现 WVBridgeDelegate 协议，供 JSBridge 回调数据给 Native。
2. 实例化自己的 WVBridge 实例，传入 env 和上面实现的 delegate。
3. 设置 WVBridge 实例的 view 和 viewController 属性，为 JSBridge API 提供环境信息。其它属性可以根据需要设置：
   - pageId：当前页面的 ID，变化后前一页面未调用的 JSBridge 会全部失效，避免影响到当前页面
   - openPermissionCheck：启用白名单校验，只允许符合 WindVane domain 配置中的 Ali 域名白名单的 URL 调用 JSBridge，其它 URL 的调用都会被拒绝。
4. 调用 callHandlerWithRequest:、callHandlerWithURL: 或 callHandler:withParams:withReqId: 方法之一，就会调用实际的 JSBridge；JSBridge 的回调、事件都会通过 delegate 返回回来。

标准事件机制

WindVane 的标准事件机制，提供了 Native 和 H5 之间互发事件的标准通讯接口，便于各业务场景下的基于统一的事件模型来通讯。

Native 向 H5 发送事件

WindVane 提供了 WVStandardEventCenter 类，供 Native 向 H5 发送事件：

参数中，webView 是要发送事件的目标 WebView，eventName 是事件的名称（字符串），eventData 是事件的数据（iOS 是 NSDictionary *，Android 是一个 JSON Object 格式的 String）。

在 iOS 平台的 JSBridge 中，不建议使用这里的方法来发送事件，而是使用 JSBridge context 的 dispatchEvent 方法来发送事件，对多种容器（WebView/Weex）提供了兼容。

```
#import <WindVane-Basic/WVStandardEventCenter.h>

// 向指定 WebView 发送事件。

[WVStandardEventCenter postNotificationToJS:eventName withEventData:eventData

withWebView:webView];

// 向所有 WebView 发送事件，请谨慎使用。

[WVStandardEventCenter postNotificationToJS:eventName withEventData:eventData];
```

H5 用法

```
document.addEventListener('eventName', function(data) {
 // 这里要注意，Native 传递过来的事件参数是在 data 的 param 属性中。
 alert(data.param);
}, false);
```

H5 向 Native 发送事件

H5 用法 使用 JSBridge 来发送事件，注意引入 windvane.js，详情可以参考 WVStandardEventCenter.postNotificationToNative：

```
var params = {
 event: 'eventName',
 param: {
  // 事件要传递的数据。
 }
};
window.WindVane.call('WVStandardEventCenter','postNotificationToNative', params,
function(e) {
 alert('success');
}, function(e) {
 alert('failure: ' + JSON.stringify(e));
});
```

iOS 用法 使用标准的消息中心监听事件：

```
[[NSNotificationCenter defaultCenter] addObserver:self selector:@selector(myEventListener)
name:eventName object:nil]
- (void)myEventListener:(NSNotification *)notification {
 // 事件参数可以从 notification.userInfo 中获取。
}
```

3. JS API 指南

3.1. 接入

windvane js

要使用 JSBridge API，请引入 https://g.alicdn.com/mtb/lib-windvane/3.0.0/windvane.jshttps://g.alicdn.com/mtb/lib-windvane/3.0.0/windvane.js，它提供了调用客户端接口的功能。 可在有接入 windvane 的 app 中扫描以下二维码，查看可用的 JSBridge API

lib-emas.js

接入方也可选择接入 lib-emas.js，对 windVane 进行了封装

安装

lib-emas.js EMAS H5 容器前端 JSAPI 适配层，改善原有 windvane 调用方式

业务前端不需要再引入原来的 windvane.js，直接依赖本库（cdn 方式 TODO）

```
npm install --save lib-emas
```

调整 jsapi 调用方式为链式调用，使用更加方便

原 windvane 方式调用 JSAPI

```
let params = {
    phone: '18651836022'
}
window.WindVane.call('Call', 'call', params, function(e) {
  }, function(e) {
    alert('failure' + JSON.stringify(e));
  });
```

优化后调用方式

```
let params = {
    phone: '18651836022',
    onSuccess: function(data){},
    onFail: function(err){}
}
emas.Call.call(params)
```

支持 promise(推荐)

```
emas.Call.call(params).then(function(data){}).catch(function(err){})
```

提供横向扩展

业务可以自己撰写原生代码扩展自定义 JSAPI 后，可以在这里同步扩展下调用层

比如业务扩展了一个类叫 Path，有一个方法叫 method

注册方法

```
emas.add("Path.method")
```

使用方法

```
emas.Path.method(params)
```

3.2. 规范与约束

UA 规范

WindVane 会在 所设置的默认 UA 的后面跟上 WindVane 标记、WindVaneSDK 的版本号、客户端补充 UA 和当前屏幕的分辨率，便于通过判断 UA 的方式来检查环境。完整的 UA 格式为：

默认 UA（App 名称/App 版本号）WindVane/WindVaneSDK 版本号 客户端补充 UA 分辨率宽 x 分辨率高

- WindVane iOS 需要通过 setAppUA 方法自行设置客户端补充 UA，且分辨率中使用的是小写字母 x；
- WindVane Android 没有提供设置客户端补充 UA 的方法，分辨率中使用的是大写字母 X。
- Win10 桌面版中，由于只能取到窗口大小（会发生变化），因此未加入分辨率。

__警告：__H5 开发者不要将客户端补充 UA 作为识别 iOS/Android 平台，或客户端版本的依据。

可以使用 amfe-env 库来做环境判断，或者通过 JS 自行判断环境：

示例：判断是否接入了 WindVane

```
alert('获取到客户端的 UA: ' + window.navigator.userAgent);

if (window.navigator.userAgent.match(/WindVane/i)) {
    alert('当前运行环境为接入了 WindVane Framework 的客户端 WebView');
}
```

示例：判断手机平台

```
alert('获取到客户端的 UA: ' + window.navigator.userAgent);

if ((/Windows\s(Phone(?:\sOS)?|NT)\s[\d\.]+/i).test(window.navigator.userAgent)) {
 alert('当前运行环境是 Windows 客户端');
} else if ((/iPhone|iPad|iPod/i).test(window.navigator.userAgent)) {
 alert('当前运行环境是 iOS 客户端');
} else if ((/Android/i).test(window.navigator.userAgent)) {
 // 注意：由于 Windows Phone 的 UA 中也包含 Android 字符串，因此独立使用时注意兼容问题。
 alert('当前运行环境是 Android 客户端');
}
```

示例：判断 UC 内核

```
alert('获取到客户端的 UA: ' + window.navigator.userAgent);

if ((/ UCBrowser\//).test(window.navigator.userAgent)) {
 if ((/ U3\//).test(window.navigator.userAgent)) {
  alert('当前运行环境是 U3 内核');
 } else if ((/ UCBS\//).test(window.navigator.userAgent)) {
  alert('当前运行环境是 U4 内核');
 } else {
  alert('当前运行环境是 UC 内核');
 }
}
} else {
 alert('当前运行环境不是 UC 内核');
}
```

WebGL 使用规范

iOS 平台下，若__客户端已退到后台，但是 HTML 中仍在使用 webgl 进行 canvas 渲染，那么就会导致客户端 Crash__。Crash 堆栈中一般会带有 _gpus_ReturnNotPermittedKillClient 或类似的堆栈。

原因是 iOS 下的 UIWebView 使用 webgl 渲染时，WebCore 会调用到 OpenGL ES 进行渲染，而苹果发现有在后台调用 OpenGL ES，就会直接结束 App。相关文档可以参考这里。

所以前端禁止在 App 退到后台后做任何 webgl 操作，我们提供了以下四种解决方案：

1. 切换到 WKWebView。
2. 使用 @ali/game-loop 或类似前端库，实现生命周期管理，内部封装了下面提到的切换前后台事件。
3. 在需要做 webgl 渲染时，调用 JSBridge WVApplication.appState 判断 App 状态，如果不是 'active'，那么说明 App 不在前台，不能够做 webgl 渲染。由于 JSBridge 的调用有消耗，所以适合于偶尔渲染一次的情况。
4. 监听 WindVane 提供的切换前后台事件，在事件中主动暂停/恢复 webgl 的渲染。适合一直在进行绘制的情况，可以参考以下代码：

```
document.addEventListener('WV.Event.APP.Background', function(e) {
 if (e.param.to && e.param.to === 'background') {
  // 退到后台，暂停 webgl 渲染操作。
 }
}, false);
document.addEventListener('WV.Event.APP.Lock', function(e) {
 // 触发锁屏，暂停 webgl 渲染操作。
}, false);
document.addEventListener('WV.Event.APP.Active', function(e) {
 if (e.param.from && e.param.from === 'background') {
  // 回到前台，恢复 webgl 渲染操作。
 }
}, false);
```

注意 WV.Event.APP.Background 和 WV.Event.APP.Lock 事件可能会都被触发，所以逻辑上请处理好同时触发的情况。

警告：在后台执行 getContext('webgl') 就可能会产生 Crash，原因是 WebView 会在初始化 *WebGLRenderingContext* 时调用 OpenGL ES 做一次 reshape，就会导致 Crash 产生。因此要注意 getContext('webgl') 的时机，如果可以的话最好用 WVApplication.appState 判断一下 App 状态。

https 使用规范

__注意：在 WWDC 2016 开发者大会上，苹果宣布 2017 年 1 月 1 日起，苹果 App Store 中的所有 App 都__必须启用 App Transport Security (ATS)。

因此 iOS 平台的 App，需要使用 https 发请求，**包括 H5 页面，以及页面加载的任何资源求也同样需要切换为 https，请前端同学务必重视这件事情！**

目前苹果为 iOS 客户端提供了以下配置，来对 http 协议做兼容，但都有很严格的审核限制，会强制触发 AppStore review 并需要提供解释，客户端未必能够配置：

1. NSAllowsArbitraryLoads，设为为 YES 的话就会允许任意的 http 请求，但是基本__无法通过苹果的审核__，所以客户端不会配置。
2. NSAllowsArbitraryLoadsInWebContent，设为为 YES 的话就会允许 UIWebView/WKWebView 发出的任意 http 请求（除了视频），但是也__存在无法通过 AppStore 审核的可能__，所以客户端未必会配置。

3. NSAllowsArbitraryLoadsForMedia，设为为 YES 的话就会允许发出任意的视频 http 请求（包括 H5 和 Native），但是要求仅当视频已使用 *FairPlay* 或 *secure HLS* 之类的技术加密的前提下才可以配置，所以客户端基本不会配置。

已知系统问题

1. 在 iOS 系统的 UIWebView 上，如果页面中同步请求了大量资源（比如同步三四十个请求），或用户频繁创建和销毁 WebView，有可能会造成 WebThread 死循环，最后导致主线程卡死。其特征是，主线程的堆栈包含：

```
0    libsystem_kernel.dylib          0x00000001975d30c0 ___psynch_mutexwait :8 (in
libsystem_kernel.dylib)
1    libsystem_pthread.dylib         0x000000019766d54c __pthread_mutex_lock :420 (in
libsystem_pthread.dylib)
2    WebCore                         0x0000000194125d84 __ZL17_WebTryThreadLockb :52 (in
WebCore)
3    WebCore                         0x0000000194126778 _WebThreadLock :104 (in WebCore)
```

且 WebThread 以下面或类似的堆栈结束，包含 WTF::HashMap 的操作：

```
thread #74: tid = 0x38cabd, 0x31382130 WebCore`WTF::HashMap<unsigned long,
WTF::RefPtr<WebCore::ResourceLoader>, WTF::IntHash<unsigned long>,
WTF::HashTraits<unsigned long>,
WTF::HashTraits<WTF::RefPtr<WebCore::ResourceLoader> > >::remove(unsigned long const&) +
104, name = 'WebThread'
```

这个问题的原因，是 WTF::HashMap 本身出现了死循环，怀疑与并发读写内存，或内存溢出有关，但尚未明确。

目前的规避方案，是减少页面同步加载的资源个数，或者使用懒加载的方案，将同步资源请求数限制在一二十个以内就不太容易出现问题。

1. 在 iOS 9.3.X 上，Web 页面中使用 MutationObserver 会偶现 Crash；前端在此版本上请不要使用 MutationObserver，或者使用其它技术代替。

   注意： WindVane iOS 会在 iOS 9.3.X 中默认引入 polyfill，将 MutationObserver 切换到 Mutation Events 的实现以避免 Crash。**但由于 iOS 系统的限制，attributes 的改变和 Contenteditable 元素的内容改变都不会触发 MutationObserver。** 如果希望继续使用 MutationObserver，可以使用 window.WebKitMutationObserver，并对 Crash 率上升做好预期。

2. 在 iOS 11.X 上，如果页面用到了 WebWorker，有可能出现以下两种 Crash：

```
Thread 21 WebThread Crashed:
0   JavaScriptCore                  0x0000000188eae188
_ZN3WTF20installSignalHandlerENS_6SignalEONS_8FunctionIFNS_12SignalActionESO_RNS_7SigInfoE
R27__darwin_arm_thread_state64EEE :700 (in JavaScriptCore)
1   JavaScriptCore                  0x0000000188e16b70
_ZNSt3__117__call_once_proxyINS_5tupleIJOZN3JSC7VMTraps12SignalSenderC1ERKN3WTF14AbstractL
ockerERNS2_2VMEEU1vE_EEEEEvPv :52 (in JavaScriptCore)
2   libc++.1.dylib                  0x000000018114c944 std::__1::__call_once(unsigned long
volatile&, void*, void (*)(void*)) :160 (in libc++.1.dylib)
3   JavaScriptCore                  0x0000000188e16904
JSC::VMTraps::SignalSender::SignalSender(WTF::AbstractLocker const&, JSC::VM&) :260 (in
JavaScriptCore)
4   JavaScriptCore                  0x0000000188e1641c
JSC::VMTraps::fireTrap(JSC::VMTraps::EventType) :240 (in JavaScriptCore)
5   WebCore                         0x000000018ab3b120 WebCore::WorkerThread::stop() :76
(in WebCore)
6   WebCore                         0x00000001899c1420
WebCore::ScriptExecutionContext::stopActiveDOMObjects() :256 (in WebCore)
7   WebCore                         0x00000001899c1778
WebCore::Document::prepareForDestruction() :564 (in WebCore)
8   WebCore                         0x0000000189e59f48
_ZN7WebCore5Frame7setViewEON3WTF6RefPtrINS_9FrameViewEEE :148 (in WebCore)
9   WebCore                         0x0000000189e656f0
WebCore::FrameLoader::detachFromParent() :520 (in WebCore)
10  WebKitLegacy                    0x000000018ade0588 __29-[WebView(WebPrivate)
_close]_block_invoke :344 (in WebKitLegacy)
```

```
Thread 24 WebThread Crashed:
0   JavaScriptCore                  0x000000018b012188
_ZN3WTF20installSignalHandlerENS_6SignalEONS_8FunctionIFNS_12SignalActionESO_RNS_7SigInfoE
R27__darwin_arm_thread_state64EEE :700 (in JavaScriptCore)
1   JavaScriptCore                  0x000000018af7ab70
_ZNSt3__117__call_once_proxyINS_5tupleIJOZN3JSC7VMTraps12SignalSenderC1ERKN3WTF14AbstractL
ockerERNS2_2VMEEU1vE_EEEEEvPv :52 (in JavaScriptCore)
2   libc++.1.dylib                  0x00000001832b0944 std::__1::__call_once(unsigned long
volatile&, void*, void (*)(void*)) :160 (in libc++.1.dylib)
3   JavaScriptCore                  0x000000018af7a904
JSC::VMTraps::SignalSender::SignalSender(WTF::AbstractLocker const&, JSC::VM&) :260 (in
JavaScriptCore)
4   JavaScriptCore                  0x000000018af7a41c
JSC::VMTraps::fireTrap(JSC::VMTraps::EventType) :240 (in JavaScriptCore)
5   WebCore                         0x000000018cc9f120 WebCore::WorkerThread::stop() :76
(in WebCore)
6   WebCore                         0x000000018c671e50
WebCore::jsWorkerPrototypeFunctionTerminate(JSC::ExecState*) :136 (in WebCore)
```

此问题可能与 JS 主动调用 WebWorker.terminate() 有关，WebView 本来也会在销毁时主动结束 WebWorker，JS 可以在 iOS11.X 上不主动销毁。

使用 WKWebView

1. 判断是 UIWebView 还是 WKWebView 环境，可以直接检查 window.webkit.messageHandlers 是否存在：

```
var isWKWebView = window.webkit && window.webkit.messageHandlers;
```

1. WKWebView 中无法通过 WVScreen.capture 截取 <canvas> 内容。

可以在截屏前将 <canvas> 内容渲染到一个 <img> 上，并将 <img> 覆盖到 <canvas> 之上实现截屏功能。

使用 JSBridge

JSBridge 极大的扩展了 H5 的能力，它提供了 H5 与客户端通讯的机制，允许 H5 调用服客户端提供的功能，并接受客户端返回的数据。如果需要通过 UA 判断客户端情况，请参考上述 UA 规范。

JSBridge 不支持在 iframe 内部调用，因为 Native 无法回调过去。

使用 window.WindVane.call(className, methodName, params, successCallback, failureCallback, timeout) 来调用客户端提供的功能。

输入参数

类型	名称	描述
string	className	要调用的客户端类名。
string	methodName	要调用的客户端方法名。
object	params	要传递给客户端的参数。
function	successCallback	执行成功后的回调。
function	failureCallback	执行失败后的回调。
number	timeout	可选 执行超时，超时后自动以 {ret:['HY_TIMEOUT']} 参数调用失败回调。

successCallback 和 failureCallback 都是由 Native 调用过来的，因此如果需要 try...catch 捕获异常，需要将捕获写到回调函数内部。

为了简便起见，文档中会将要调用的客户端类名和方法，记作 `className.methodName` 的格式。

回调参数 如果成功执行客户端接口，则进入 success 回调，否则进入 failure 回调，回调函数的参数与被调用的接口相关。

可能的回调 ret 回调参数中可能包含 ret 属性，表示此次执行的状态，其可能的值为：

值	描述
'HY_SUCCESS'	表示 API 执行成功了，该值只会在 success 回调中使用。
'HY_CLOSED'	表示 JSBridge 功能被强制关闭，一般由于页面 URL 不在信任域名内，或 Native 端未开启 JSBridge 功

	能，请检查 URL 或者联系 H5 的容器 Native 开发排查。
'HY_NO_HANDLER'	表示当前客户端没有提供这个 API，请检查调用的 JSBridge 是否正确，并联系 JSBridge 的 Native 提供方排查。
'HY_PARAM_ERR'	表示传入的参数错误，请检查传入的参数是否有误。
'HY_NO_PERMISSION'	表示当前页面没有权限调用这个接口。
'HY_FAILED'	表示 API 执行失败了，请联系 JSBridge 的 Native 提供方排查。
'HY_EXCEPTION'	表示 API 执行时发生了异常，请联系 JSBridge 的 Native 提供方排查。
'HY_USER_DENIED'	表示当前 API 缺少用户授权，例如需要访问相册的接口，但用户并未允许客户端访问相册。
'HY_USER_CANCELLED'	表示用户主动取消了当前 API 的功能执行。
'HY_RET_PHOTO_CANCLE'	表示使用 WVCamera 提供的接口拍照时，被用户取消。
'HY_TIMEOUT'	表示 JSBridge 调用超时。
'HY_RESULT_PARSE_ERROR'	表示 Native 传递给 H5 的数据出现格式错误
'HY_NOT_IN_WINDVANE'	表示当前不是 WindVane 环境（如浏览器），或者客户端未正确提供 WindVane 的 UserAgent。也有可能是页面 JS 错误的修改了 windvane.js 中的环境变量（例如设置了 WindVane.isAvailable = false）。
'HY_NOT_SUPPORT_DEVICE'	表示 JSBridge 在不支持的设备中（如浏览器）被调用。也有可能是页面 JS 错误的修改了 windvane.js 中的环境变量（例如设置了 WindVane.isAvailable = false）。

'HY_NO_HANDLER_ON_WP'	表示 JSBridge 在不支持的 Windows 设备中（如浏览器）被调用。
'WV_ERR::PARAM_PARSE_ERROR'	表示 Native 传递给 H5 的数据出现格式错误。

根据调用的 JSBridge API 的不同，ret 属性可能不存在，也**可能是一个字符串**，也可能是一个数组，其中 ret[0] 包含了客户端接口的执行状态。 并不是所有 JSBridge 都会调用回调函数，这与客户端的实现有关。JSBridge 文档中总会指出什么情况下会进入成功回调或失败回调。

下面以调用 MyJSBridge.firstAPI 为例：

```
var params = {
 'myParam': 'Something to Native',
 'myParam2': {
  'p': 'ps'
 }
};
window.WindVane.call('MyJSBridge', 'firstAPI', params, function(e) {
    alert('success' + JSON.stringify(e));
}, function(e) {
    alert('failure' + JSON.stringify(e));
});
```

3.3. 基础 Base

Base.checkWindVaneSDK

仅限 WindVane iOS

判断指定版本号与当前 WindVaneSDK 版本号的大小关系。

输入参数

- [string] v - 要判断的版本号，例如 '5.3.0'。

回调参数

回调参数将会在回调方法中传递，如果成功比较版本号，则进入 success 回调，否则进入 failure 回调。

- [string] isHigher - 指定版本号与当前 WindVaneSDK 版本号的大小关系，可能的值为：
  - '1' - 指定版本号高于当前 WindVaneSDK 版本号。

- '0' - 指定版本号与当前 WindVaneSDK 版本号相同。
- '-1' - 指定版本号低于当前 WindVaneSDK 版本号。

```
var params = {
 // 要判断的版本号
 v: '5.3.0'
};
window.WindVane.call('Base', 'checkWindVaneSDK', params, function(e) {
 alert('success: ' + JSON.stringify(e));
}, function(e) {
 alert('failure: ' + JSON.stringify(e));
});
```

Base.copyToClipboard

将指定的文本复制到剪贴板。

输入参数

- [string] text - 要复制到剪贴板中的文本。

回调参数

无回调参数，如果成功复制到剪贴板，则进入 success 回调，否则进入 failure 回调。

```
var params = {
 text: "text to copy"
};
window.WindVane.call('Base', 'copyToClipboard', params, function(e) {
 alert('success' + JSON.stringify(e));
}, function(e) {
 alert('failure' + JSON.stringify(e));
});
```

Base.isInstall

判断是否已安装指定的应用程序。

__注意：__在 iOS9 或更高版本系统中，只有在 info.plist 的 LSApplicationQueriesSchemes 中配置过的 scheme，才能够正确判断引用是否安装。

输入参数

- [string] ios - iOS 平台的应用程序标记，使用的是 schema，例如 'taobao://'。
- [string] android - Android 平台的应用程序标记，使用的是包名，例如 'com.taobao.taobao'。

回调参数

无回调参数，如果已安装指定应用程序，则进入 success 回调，否则进入 failure 回调。

```
var params = {
 // iOS 平台的应用程序标记
 ios: 'taobao://',
 // Android 平台的应用程序标记
 android: 'com.taobao.taobao'
};
window.WindVane.call('Base', 'isInstall', params, function(e) {
 alert('success');
}, function(e) {
 alert('failure');
});
```

Base.isAppsInstalled

判断是否已安装指定的多个应用程序。

注意在 iOS9 或更高版本系统中，只有在 info.plist 的 LSApplicationQueriesSchemes 中配置过的 scheme，才能够正确判断引用是否安装。

输入参数

- [object] appName - 要判断是否安装的 App，其值是一个对象，包含以下属性：
  - [string] ios - iOS 平台的应用程序标记，使用的是 schema，例如 'taobao://'。
  - [string] android - Android 平台的应用程序标记，使用的是包名，例如 'com.taobao.taobao'。

回调参数

回调参数将会在回调方法中传递，如果成功检测应用程序是否安装，则进入 success 回调，否则进入 failure 回调。

- [boolean] appName - 之前传递的 App 是否安装。

```
var params = {
 taobao: {
  ios: 'taobao://',
  android: 'com.taobao.taobao'
 },
 tmall: {
  ios: 'tmall://',
  android: 'com.tmall.tmall'
 }
};
window.WindVane.call('Base', 'isAppsInstalled', params, function(e) {
 alert('success: ' + JSON.stringify(e));
}, function(e) {
 alert('failure: ' + JSON.stringify(e));
});
```

Base.isWindVaneSDK

用于获取当前 WindVaneSDK 的版本，该参数的替代方法是自己从 UA 中获取到 WindVane 的 SDK 版本号。

输入参数

无输入参数。

回调参数

回调参数将会在回调方法中传递，如果成功获取 WindVaneSDK 的版本，则进入 success 回调，否则进入 failure 回调。

- [string] os - 平台类型（'ios'/'android'）。
- [string] version - WindVaneSDK 的版本号。
- [string] env - 客户端环境（'release'/'pre'/'daily'）VER.WindVane iOS 8.1.0 或更高。
- [string] container - 容器类型（'WVWebView'/'WVWKWebView'/'WVUCWebView'/'Weex'）VER.WindVane 8.1.0 或更高。

```
window.WindVane.call('Base', 'isWindVaneSDK', {}, function(e) {
 alert('success: ' + JSON.stringify(e));
}, function(e) {
 alert('failure: ' + JSON.stringify(e));
});
```

Base.notify

仅限 WindVane iOS

在 document 引发指定名称和参数的事件。

输入参数

- [string] event - 要引发的事件名称。
- [object] param - 要引发的事件参数。

回调参数

无回调参数，也不会进入 success 或 failure 回调。

监听事件

由 event 指定的事件，事件参数为 param 指定的对象。

```
document.addEventListener('MyNotifyName', function(e) {
 alert(JSON.stringify(e.param));
}, false);

var params = {
 // 要引发的事件名称
 event: 'MyNotifyName',
 // 要引发的事件参数
 param: { MyEventParam: 'paramValue' }
};
window.WindVane.call('Base', 'notify', params);
```

Base.openBrowser

仅限 WindVane iOS

打开指定 URL 的 Safari 新窗口。

输入参数

- [string] url - 要在 Safari 中显示的 URL。

__注意：__仅需要确保在 Safari 中打开的 URL，才使用此 JSBridge。其它情况都请直接使用 <a> 链接、设置 location.href 或者 iframe 的方式进行跳转。

回调参数

无回调参数，如果成功打开了指定 URL，则进入 success 回调，否则进入 failure 回调。

```
var params = {
 // 要在浏览器中显示的 URL
 url: 'http://www.baidu.com'
};
window.WindVane.call('Base', 'openBrowser', params, function(e) {
}, function(e) {
 alert('failure: ' + JSON.stringify(e));
});
```

Base.setBackgroundColor

仅限 WindVane iOS

设置 WebView 的背景颜色。

输入参数

- [string] color - WebView 背景颜色的十六进制颜色值，支持 0X 或 # 前缀。
- [number] alpha - *[可选]*WebView 背景不透明度，有效范围为 [0, 1]。

回调参数

无回调参数，如果成功设置背景透明度，则进入 success 回调，否则进入 failure 回调。

```
var params = {
 color: 'FF0000',
 alpha: 0.3
};
```

```
window.WindVane.call('Base', 'setBackgroundColor', params, function(e) {
 alert('success: ' + JSON.stringify(e));
}, function(e) {
 alert('failure: ' + JSON.stringify(e));
});
```

3.4. 窗口小部件 Widget

WVUIActionSheet.show

显示一个 ActionSheet（显示在屏幕下方的动作列表）。

iOS	Android 效果

输入参数

- [string] title - ActionSheet 的标题。
- [int] _index - ActionSheet 的索引，用于唯一标识该 ActionSheet。
- [array] buttons - ActionSheet 中显示的按钮，每一项都是一个字符串，表示按钮的标题。*ActionSheet 最后总是有一个额外的按钮：取消。*

回调参数

无回调参数，如果成功显示 ActionSheet，则进入 success 回调，否则进入 failure 回调。

监听事件

wv.actionsheet 用户响应了 ActionSheet。

事件参数：

- [string] type - 用户点击的按钮文本。
- [int] _index - ActionSheet 的索引。

__注意：__在 iOS 平台下，用户点击取消按钮，或者在 ActionSheet 外点击，都会引发 type:'取消' 的事件；而在 Android 平台下，用户点击取消按钮会引发不含 type 属性的事件，在 ActionSheet 外点击则不会引发任何事件。

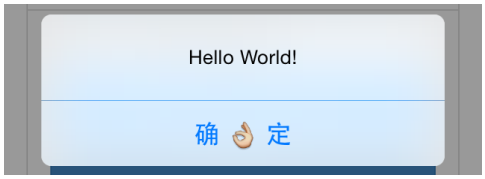
```
document.addEventListener('wv.actionsheet', function(e) {
 alert(JSON.stringify(e.param));
}, false);


var params = {
 // ActionSheet 的标题
 title: 'Choose a button!',
 // ActionSheet 的索引
 _index: 32768,
 // ActionSheet 中显示的按钮
 buttons: ['按钮 1', '按钮 2', '按钮 3', '按钮 4', '按钮 5']
};
window.WindVane.call('WVUIActionSheet', 'show', params, function(e) {
}, function(e) {
 alert('failure: ' + JSON.stringify(e));
});
```

WVUIDialog.alert

显示一个警告框。

iOS 效果	Android 效果

输入参数

- [string] message - 警告框要显示的消息。
- [string] okbutton - 警告框的确认按钮文本。
- [string] identifier - 警告框的标识符。

监听事件

WV.Event.Alert 用户响应了警告框。

事件参数：

- [string] identifier - 警告框的标识符。

回调参数

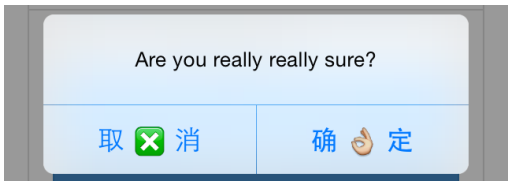
无回调参数，如果成功显示警告框，则进入 success 回调，否则进入 failure 回调。

```
var params = {
 // 警告框要显示的消息
 message: 'Hello World!',
 // 警告框的确认按钮文本
 okbutton: '确 定'
};
window.WindVane.call('WVUIDialog', 'alert', params, function(e) {
}, function(e) {
 alert('failure: ' + JSON.stringify(e));
});
```

WVUIDialog.confirm

显示一个确认框。

iOS 效果	Android 效果



输入参数

- [string] message - 确认框要显示的消息。
- [string] okbutton - 确认框的确认按钮文本。
- [string] canclebutton - 确认框的取消按钮文本。
- [int] _index - 确认框的索引，用于唯一标识该确认框。

回调参数

无回调参数，如果成功显示警告框，则进入 success 回调，否则进入 failure 回调。

监听事件

wv.dialog 用户响应了确认框。

事件参数：

- [string] type - 用户点击的按钮文本，可能是输入参数中的 okbutton 或 canclebutton。
- [int] _index - 确认框的索引，与输入参数中的 _index 相同。

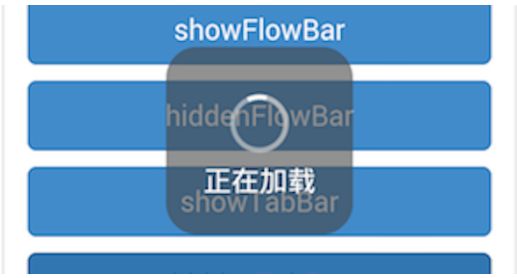
```
document.addEventListener('wv.dialog', function(e) {
alert(JSON.stringify(e.param));
}, false);

var params = {
// 确认框要显示的消息
message: 'Are you really really sure?',
// 确认框的确认按钮文本
 okbutton: '确定',
// 确认框的取消按钮文本
canclebutton: '取消',
// 确认框的索引
```

```
_index: 10086
};
window.WindVane.call('WVUIDialog', 'confirm', params, function(e) {
}, function(e) {
alert('failure: ' + JSON.stringify(e));
});
```

WVUI.showLoadingBox

显示一个加载提示框。

iOS 效果	Android 效果
	

输入参数

无输入参数。

回调参数

无回调参数，如果成功显示加载提示框，则进入 success 回调，否则进入 failure 回调。

```
window.WindVane.call('WVUI', 'showLoadingBox', {}, function(e) {
}, function(e) {
 alert('failure: ' + JSON.stringify(e));
});
```

WVUI.hideLoadingBox

隐藏加载提示框。

输入参数

无输入参数。

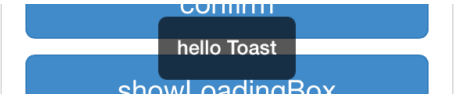
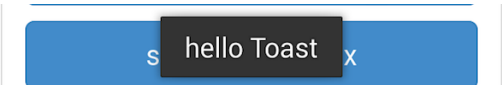
回调参数

无回调参数，如果成功隐藏警告框，则进入 success 回调，否则进入 failure 回调。

```
window.WindVane.call('WVUI', 'hideLoadingBox', {}, function(e) {
},  function(e) {
 alert('failure: ' + JSON.stringify(e));
});
```

WVUIToast.toast

显示一个 Toast（显示在屏幕中央的简单提示）。

iOS 效果	Android 效果
	

输入参数

- [string] message - Toast 要显示的消息。
- [number] duration - *[可选]*Toast 的持续时间（秒），默认为 0.8。

回调参数

无回调参数，如果成功显示 Toast，则在 Toast 显示完毕后进入 success 回调；否则进入 failure 回调。

```
var params = {
 // Toast 要显示的消息
 message: 'Toast information',
 // Toast 的持续时间
 duration: 5
};
window.WindVane.call('WVUIToast', 'toast', params, function(e) {
},  function(e) {
```

```
  alert('failure: ' + JSON.stringify(e));
});
```

3.5. 声音播放 WVAudio

WVAudio.playSystemSound

仅限 WindVane iOS

播放指定的系统声音。

输入参数

- [int] sound - 要播放的系统声音 ID，iOS 系统下的声音 ID 可以参考 AudioServices。
- [int] count - 声音连续播放的次数。

回调参数

无回调参数，如果声音播放成功，则进入 success 回调，否则进入 failure 回调。

```
var params = {
  // 要播放的系统声音 ID
  sound: 1000
  // 声音连续播放的次数
  count: 3
};
window.WindVane.call('WVAudio', 'playSystemSound', params, function(e) {
  alert('success');
}, function(e) {
  alert('failure: ' + JSON.stringify(e));
});
```

3.6. 相机 WVCamera

WVCamera.takePhoto

用户自选调用相机拍照或者直接从相册选择照片，得到照片后可以指定是否自动上传照片。需要 native 方实现上传协议。

__注意：__在 iOS 11 中，要求客户端添加一个新的权限描述 NSPhotoLibraryAddUsageDescription，才允许访问相册。

示例代码如下所示：

```
var photoLocalPath; // 保存本地文件的路径。
var params = {
 type: '0',
};
window.WindVane.call('WVCamera', 'takePhoto', params, function(e) {
 var uploadParams = {
  // 需要上传的照片的路径
  path: e.localPath
 };
 setTimeout(function() {
  window.WindVane.call('WVCamera', 'confirmUploadPhoto', uploadParams, function(e) {
   alert('upload success: ' + JSON.stringify(e));
  }, function(e) {
   alert('upload failure: ' + JSON.stringify(e));
  });
 }, 20);
}, function(e) {
 alert('takePhoto failure: ' + JSON.stringify(e));
});
```

输入参数自定义

回调参数

回调参数将会在回调方法中传递，如果成功获取照片，则进入 success 回调；如果失败或被用户取消，则进入 failure 回调。

成功回调参数：

- [string] url - 访问 URL，页面将此 URL 填写给 img 的 src 属性，用于进行照片的预览。 **注意：该 URL 非真实的 CDN URL，浏览器无法访问。**
- [string] localPath - 照片的本地文件路径，该路径可以用于后续的 WVCamera.confirmUploadPhoto 上传过程。

- [string] resourceURL - 上传到 TFS 后的回传 CDN 地址，该地址可以在浏览器里真实访问。仅当照片被成功上传到 CDN 之后才存在该属性。
- [string] identifier - 照片标识，当使用 Mtop 上传照片（v = '2.0'）时，会将输入参数中的 identifier 返回。
- [string] base64Data - 照片的 Base64 数据，仅当设置了 needBase64 = true 时才有效，实际使用时请注意自行添加 前缀（VER.WindVane 8.0.0 或更高）。

失败回调参数：

- [string] errorType - 错误类型。
- [string] errorCode - 错误代码。
- [string] errorMsg - 错误消息。
- [string] identifier - 照片标识，当使用 Mtop 上传照片（v = '2.0'）时，会将输入参数中的 identifier 返回。

__注意：__如果用户手动取消了拍照，会进入失败回调，iOS 的参数是 {ret:'HY_RET_PHOTO_CANCLE'}，Android 的参数是 {ret:'HY_FAILED'}，没有其它参数。如果没有访问相机/相册的权限，同样会进入失败回调，并且参数都会包含 {msg:'NO_PERMISSION'}。

监听事件

WVPhoto.Event.takePhotoSuccess 获取照片成功，即将上传，此时 JS 可以在页面上对照片进行预览。

事件参数：

- [string] url - 访问 URL，页面将此 URL 填写给 img 的 src 属性，用于进行页面的预览。 **注意：该 URL 非真实的 CDN URL，浏览器无法访问。**
- [string] localPath - 本地文件路径，该路径可以用于后续的上传过程。

多图选择模式

允许同时选择多张照片上传，仅限 WindVane iOS，而且只有从相册选择照片时才能够选择多张，直接拍照时仍会使用上面的单张照片模式，因此建议传入参数 mode: 'photo' 以直接进入相册。

__注意：__Android 目前只有在手淘中才能使用多图选择模式，而且在 VER.手淘 Android 0330 之前，快速滑动时存在线程溢出的 BUG，因此建议从 VER.0330 之后再使用。

输入参数

- [string] mutipleSelection - *[可选]*是否使用多图选择模式，'1' 表示使用多图选择模式，'0' 表示不使用多图选择模式，默认为 '0'。
- [int] maxSelect - *[可选]*多图选择模式下最多选择的照片数，默认为 9。

回调参数

回调参数将会在回调方法中传递，如果成功获取，则进入 success 回调；如果失败或被用户取消，则进入 failure 回调。

成功回调参数：

- [array] images - 用户选择的照片数组，每一项包含以下属性：

- [string] url - 访问 URL，页面将此 URL 填写给 img 的 src 属性，用于进行照片的预览。 **注意：该 URL 非真实的 CDN URL，浏览器无法访问。**
- [string] localPath - 照片的本地文件路径，该路径可以用于后续的 WVCamera.confirmUploadPhoto 上传过程。

失败回调参数：

- [string] errorType - 错误类型。
- [string] errorCode - 错误代码。
- [string] errorMsg - 错误消息。
- [string] identifier - 照片标识，当使用 Mtop 上传照片（v = '2.0'）时，会将输入参数中的 identifier 返回。

__注意：__如果用户手动取消了拍照，会进入失败回调，iOS 的参数是 {ret:'HY_RET_PHOTO_CANCLE'}，Android 的参数是 {ret:'HY_FAILED'}，没有其它参数。如果没有访问相机/相册的权限，同样会进入失败回调，并且参数都会包含 {msg:'NO_PERMISSION'}。

监听事件

WVPhoto.Event.takePhotoSuccess 获取照片成功，即将上传，此时 JS 可以在页面上对照片进行预览。

事件参数：

- [string] url - 访问 URL，页面将此 URL 填写给 img 的 src 属性，用于进行页面的预览。 **注意：该 URL 非真实的 CDN URL，浏览器无法访问。**
- [string] localPath - 本地文件路径，该路径可以用于后续的上传过程（仅限 WindVane Android）。

下列两个事件仅当拍照完直接上传才会触发：

WVPhoto.Event.uploadPhotoSuccess 上传照片成功，每上传一张照片成功都会触发此事件。

事件参数：

- [string] url - 访问 URL，页面将此 URL 填写给 img 的 src 属性，用于进行照片的预览。 **注意：该 URL 非真实的 CDN URL，浏览器无法访问。**
- [string] localPath - 照片的本地文件路径，该路径可以用于后续的 WVCamera.confirmUploadPhoto 上传过程。
- [string] resourceURL - 上传到 TFS 后的回传 CDN 地址，该地址可以在浏览器里真实访问。仅当照片被成功上传到 CDN 之后才存在该属性。
- [string] identifier - 照片标识，当使用 Mtop 上传照片（v = '2.0'）时，会将输入参数中的 identifier 返回。

WVPhoto.Event.uploadPhotoFailed 上传照片失败，每上传一张照片失败都会触发此事件。

事件参数：

- [string] errorType - 错误类型。
- [string] errorCode - 错误代码。

- [string] errorMsg - 错误消息。
- [string] identifier - 照片标识，当使用 Mtop 上传照片（v = '2.0'）时，会将输入参数中的 identifier 返回。

```
document.addEventListener('WVPhoto.Event.uploadPhotoSuccess', function (e) {
 alert('event uploadPhotoSuccess: ' + JSON.stringify(e.param));
});
document.addEventListener('WVPhoto.Event.uploadPhotoFailed', function (e) {
 alert('event uploadPhotoFailed: ' + JSON.stringify(e.param));
});


var params = {
 // 得到照片后是否自动上传
 type: '1',
 // 是否只允许拍照或从相册选择
 mode: 'photo',
 // 上传方式
 v: '2.0',
 // 业务代码
 bizCode: 'mtopupload',
 // 是否使用多图选择模式
 mutipleSelection: '1',
 // 多图选择模式下最多选择的照片数
 maxSelect: 6
};
window.WindVane.call('WVCamera', 'takePhoto', params, function(e) {
 alert('takePhoto success: ' + JSON.stringify(e));
}, function(e) {
 alert('takePhoto failure: ' + JSON.stringify(e));
});
```

WVCamera.confirmUploadPhoto

确认上传照片

输入参数

- [string] path - 需要上传的照片的路径，必须为 takePhoto 方法返回的 localPath 字段的值。
- [string] v - 得到照片后的上传方式，建议总是传入 '2.0'，表示使用 Mtop 上传（需要设置 bizCode 属性，例如 'mtopupload'。特别提示：v: '2.0' 不能简写为 v: '2'），其它上传方式将被废弃。
- [string] bizCode - 业务代码，用于标识自己的业务点。
- [string] identifier - *[可选]*照片标识，用于鉴别每次上传的照片，仅当使用 Mtop 上传照片（v = '2.0'）时才有效。
- [string] mode - *[可选]*是否只允许拍照或从相册选择，'camera' 表示直接拍照，'photo' 表示直接从相册选择，'both' 表示可以由用户自选拍照还是从相册选择。默认为 'both'（VER.WindVane 5.3.0 或更高）。
- [boolean] needLogin - *[可选]*使用 Mtop 上传照片（v = '2.0'）时是否要求必须登录才能上传，否则无需登录即可上传照片（VER.WindVane 6.5.0 或更高，旧版本仍然要求登录，新版本已默认无需登录）。

注意：v、bizCode 和 identifier 这三个参数，WindVane Android 在 VER.6.0.0 或更高才支持。

__注意：__上传文件之前需要在 Media Center 申请一个 bizCode 以标识自己的业务点，并易于对本业务的图片进行管理。可以使用通用的 bizCode：mtopupload 用于测试，如果需要申请 bizCode 请联系：逐风，家麒。如果要使用 v='2.0' 的上传，还需要将 业务描述、图片大小、图片格式、每天上传多少、图片空间申请的 bizCode 邮件给 @佑明，配置图片上传的通道。

__警告：__要求尽快切换到 v: '2.0' 的上传方式，其它上传方式由于 https 改造与安全性缘故将被废弃。

回调参数

回调参数将会在回调方法中传递，如果成功获取，则进入 success 回调；如果失败或被用户取消，则进入 failure 回调。

成功回调参数：

- [string] localPath - 照片的本地文件路径。
- [string] resourceURL - 上传到 TFS 后的回传 CDN 地址，该地址可以在浏览器里真实访问。

```
var params = {
  // 需要上传的照片的路径
  path: photoLocalPath,
  // 上传方式
  v: '2.0',
```

```
  // 业务代码
  bizCode: 'mtopupload'
 };
 window.WindVane.call('WVCamera', 'confirmUploadPhoto', params, function(e) {
  alert('success: ' + JSON.stringify(e));
 }, function(e) {
  alert('failure: ' + JSON.stringify(e));
 });
```

3.7. 通讯录 WVContacts

WVContacts.askAuth

仅限 WindVane iOS

请求访问通讯录的权限。

输入参数

无输入参数。

回调参数

回调参数将会在回调方法中传递，如果 iOS 版本低于 iOS 6，则无需请求通讯录权限，不会进入任何回调，否则总是进入 success 回调。

- [int] isAuthed - 是否获得了访问通讯录的权限，0 表示没有权限，1 表示有权限。

```
window.WindVane.call('WVContacts', 'askAuth', {}, function(e) {
 alert(JSON.stringify(e));
});
```

WVContacts.authStatus

获取当前对于通讯录的访问权限状态。

输入参数

无输入参数。

回调参数

回调参数将会在回调方法中传递，总是进入 success 回调。

- [boolean] isAuthed - 是否获得了访问通讯录的权限，0 表示没有权限，1 表示有权限。

- [int] status - （仅限 WindVane iOS）对于 iOS 平台，会有更详细的状态信息：
  - 0 - Not Determined。
  - 1 - Restricted。
  - 2 - Denied。
  - 3 - Authorized。

```
window.WindVane.call('WVContacts', 'authStatus', {}, function(e) {
 alert(JSON.stringify(e));
});
```

WVContacts.choose

拉起通讯录列表，用户选择一个联系人之后将联系人的名字和电话返回给 H5。

输入参数

无输入参数。

回调参数

回调参数将会在回调方法中传递，如果用户选择了联系人，则进入 success 回调，否则进入 failure 回调。

- [string] name - 选择的联系人的姓名。
- [string] phone - 选择的联系人的电话号码。

```
window.WindVane.call('WVContacts', 'choose', {}, function(e) {
 alert('success: ' + JSON.stringify(e));
}, function(e) {
 alert('failure: ' + JSON.stringify(e))
});
```

WVContacts.find

通过指定的名字和电话号码查找通讯录。

输入参数

- [object] filter - 联系人的过滤器，包含以下属性：
  - [string] name - 查找具有指定姓名的联系人。
  - [string] phone - 查找具有指定电话号码的联系人。

如果 phone 和 name 都存在则表示关联查找（同时满足两个条件）。若某个联系人有多个电话号码，会在结果列表中显示为多条记录。

__注意：__在 iOS 平台下，只要联系人姓名中包含 name ，就认为该联系人满足条件，电话号码也同理。而在 Android 平台下，5.7.8 版本之前联系人姓名必须与 name 相同，才认为该联系人满足条件，5.7.8 版本及之后版本与 iOS 相同。

回调参数

回调参数将会在回调方法中传递，总是进入 success 回调。

- [array] contacts - 满足过滤条件的联系人列表，每一项包含以下属性：
  - [string] name - 联系人的姓名。
  - [string] phone - 联系人的电话号码。

```
var params = {
 // 联系人的过滤器
 filter: {
  // 查找具有指定姓名的联系人
  name: '张三',
  // 查找具有指定电话号码的联系人
  phone: '123456'
 }
}
window.WindVane.call('WVContacts', 'find', params, function(e) {
 alert(JSON.stringify(e));
});
```

3.8. 文件 WVFile

WVFile.write

将指定的文件内容写入磁盘。

输入参数

- [string] mode - 写入文件的模式。
  - write - 表示写文件，若文件不存在则创建文件，若文件已存在则报错：error:FILE_EXIST。
  - append - 表示在文件末尾追加内容，若文件不存在则创建文件。
  - overwrite - 表示覆盖文件，文件不存在则创建文件，如果文件存在则覆盖之前文件的内容。

- [string] data - 待存入文件的内容，文件会存储在 WindVane 缓存文件路径/fileName，如果路径不存在，则自动创建路径。
- [string] fileName - 文件名，如果文件不存在，则创建文件（不允许包含"/"）。
- [string] share - 是否允许共享。

回调参数

回调参数将会在回调方法中传递，如果写入文件成功，则进入 success 回调，否则进入 failure 回调。

```
var params = {
 mode: 'overwrite',
 data: 'Hello World!!!\n',
 fileName: 'testFile.txt',
 share: 'false'
};
function writeFile () {
 window.WindVane.call('WVFile', 'write', params, function(e) {
  alert('success: ' + JSON.stringify(e));
 }, function(e) {
  alert('failure: ' + JSON.stringify(e));
 });
}
```

WVFile.read

读取指定文件的内容。

输入参数

- [string] fileName - 要读取的文件名称(不允许包含"/")，文件储存路径为 WindVane 缓存文件路径/fileName。如果路径不存在，则返回失败：error:PATH_NOT_FOUND；如果文件不存在，则返回失败：error:FILE_NOT_FOUND。

回调参数

回调参数将会在回调方法中传递，如果读取文件成功，则进入 success 回调，否则进入 failure 回调。

- [string] data - 读取到的文件内容。

```
var params = {
 fileName: 'testFile.txt',
 share: 'false'
};
function readFile () {
 window.WindVane.call('WVFile', 'read', params, function(e) {
  success(JSON.stringify(e));
 }, function(e) {
  failure(JSON.stringify(e));
 });
}
```

3.9. 跨域 Cookie WVCookie

__警告：__该 API 不确保执行成功，使用请谨慎。

WVCookie.read

读取指定 URL 的所有 Cookie。总是建议使用 JS 来读取 Cookie，只有需要读取其它域名 Cookie 时才使用此方法。

输入参数

- [string] url - 要读取 Cookie 的 URL。

回调参数

回调参数将会在回调方法中传递，如果成功获取指定 URL 的 Cookie，则进入 success 回调，否则进入 failure 回调。

- [object] value - 指定 URL 的 Cookie，其中以键值对的方式存储了所有 Cookie。

```
var params = {
 // 要读取 Cookie 的 URL
 url: 'http://h5.m.taobao.com'
};
window.WindVane.call('WVCookie', 'read', params, function(e) {
 alert('success: ' + JSON.stringify(e));
```

```
}, function(e) {
 alert('failure: ' + JSON.stringify(e));
});
```

WVCookie.write

写入指定的 Cookie。总是建议使用 JS 来写入 Cookie，只有需要向其它域名写入 Cookie 时才使用此方法。

输入参数

- [string] 要写入的 Cookie 的键 - 要写入的 Cookie，可以参考下面的示例。
- [string] domain - 要写入的 Cookie 的域。
- [string] max-age - *[可选]*要写入的 Cookie 的 max-age。
- [string] path - *[可选]*要写入的 Cookie 的 path，默认为 '/'。

__注意：__其它 Cookie 属性也可以按照 max-age 和 path 的方式额外添加进来，一次只能设置一条 Cookie，设置多条的话请多次调用该接口。

回调参数

无回调参数，如果成功写入指定 Cookie，则进入 success 回调，否则进入 failure 回调。

```
// 要写入的 Cookie，其名为 cookieKey，值为 cookieValue。
var params = {
 cookieKey: 'cookieValue',
 domain: 'h5.m.taobao.com',
};
window.WindVane.call('WVCookie', 'write', params, function(e) {
 alert('success');
}, function(e) {
 alert('failure: ' + JSON.stringify(e));
});
```

3.10. 地理位置 WVLocation

WVLocation.getLocation

获取当前的地理位置。

注意：VER.iOS 8 后苹果修改了应用的定位权限，应用必须在 Info.plist 中加入 NSLocationWhenInUseUsageDescription 和 NSLocationAlwaysUsageDescription 的关键字，才能正常的弹出请求定位权限的提示。 VER.iOS 11 后必须在 Info.plist 中同时加入 NSLocationAlwaysAndWhenInUseUsageDescription 和 NSLocationWhenInUseUsageDescription 的关键字。

输入参数

- [string] enableHighAcuracy - *[可选]*是否获取高精度的位置，true/faluse，默认为 false。
- [boolean] address - *[可选]*是否获取地址描述，如中国杭州市，true/false，如果不获取地址描述，则可以更快的返回经纬度。默认为 false。

回调参数

回调参数将会在回调方法中传递，如果成功获取，则进入 success 回调，否则进入 failure 回调。

- [object] coords - 经纬度信息：
  - [string] longitude - 经度。
  - [string] latitude - 纬度。
  - [string] accuracy - 水平精度。
- [object] address - 地址信息：
  - [string] city - 城市。
  - [string] province - 省。
  - [string] area - 区。
  - [string] road - 道路。
  - [string] addressLine - 详细地址。
  - [string] cityCode - 城市代码。如果是 Android 平台的非手淘客户端，无法获取此字段，原因是 Google 返回的数据中没有该字段。

__注意：__由于定位的精度和使用的接口的缘故，地址信息可能并不完整（例如缺少某些项），或精度较低，因此 H5 端请务必做好数据的异常处理。

```
var params = {
 // 是否获取高精度的位置
 enableHighAcuracy: true,
 // 是否获取地址描述
 address: true
};
window.WindVane.call('WVLocation', 'getLocation', params, function(e) {
 alert('success:' + JSON.stringify(e));
```

```
}, function(e) {
 alert('failure:' + JSON.stringify(e));
});
```

WVLocation.searchLocation

仅限 WindVane iOS

搜索给定地址的地理位置经纬度。

输入参数

- [string] addrs - 要搜索的地址。

回调参数

回调参数将会在回调方法中传递，如果成功获取，则进入 success 回调，否则进入 failure 回调。

- [string] longitude - 经度。
- [string] latitude - 纬度。

```
var params = {
 // 要搜索的地址
 addrs: '浙江省杭州市余杭区文一西路 960 号'
};
window.WindVane.call('WVLocation', 'searchLocation', params, function(e) {
 alert('success:' + JSON.stringify(e));
}, function(e) {
 alert('failure:' + JSON.stringify(e));
});
```

3.11. 网络检测 WVNetwork

WVNetwork.getNetworkType

获取当前网络类型。

输入参数

- [boolean] wifiStatus - 是否获取连接的 WIFI 信息（仅在使用 WIFI 时有效，会额外添加 ssid 和 bssid 返回值，WindVane 8.2.0 或更高版本）。

- [boolean] wifiList - 是否获取附近的 WIFI 信息列表（会额外添加 wifiList 返回值，WindVane Android 8.2.0 或更高版本）。

回调参数

回调参数将会在回调方法中传递，总是进入 success 回调。

- [string] type - 当前网络的类型，其可能的值为：
  - 'NONE' - 表示当前无网络。
  - 'WIFI' - 表示当前使用 WiFi。
  - 'CELL' - 表示当前使用蜂窝网络，但无法区分是 2G/3G/4G。
  - '2G' - 表示当前使用 2G 网络。
  - '3G' - 表示当前使用 3G 网络。
  - '4G' - 表示当前使用 4G 网络。
  - 'UNKNOWN' - 表示当前使用的网络无法判断。
- [string] message - 具体的网络制式，可能不存在（未知网络制式）。
- [string] ssid - 当前连接的 WIFI SSID（WindVane 8.2.0 或更高版本）。
- [string] bssid - 当前链接的 WIFI BSSID（WindVane 8.2.0 或更高版本）。
- [Array] wifiList - 附近的 WIFI 信息列表（WindVane Android 8.2.0 或更高版本），iOS 平台不支持此功能，数组的每一项都是一个对象，包含以下属性：
  - [string] ssid - WIFI 的 SSID。
  - [string] bssid - WIFI 的 BSSID。

```
window.WindVane.call('WVNetwork', 'getNetworkType', {}, function(e) {
 alert(JSON.stringify(e));
});
```

3.12. 动作检测 WVMotion

WVMotion.listenBlow

开启吹气动作的监听。

输入参数

- [number] time - *[可选]*吹气事件的时间间隔（秒），在用户吹气的过程中，会每隔指定时间引发一次吹气事件，默认为 0，由系统自动决定（仅限 WindVane iOS）。

回调参数

无回调参数，如果成功开启了吹气的监听，则进入 success 回调，否则进入 failure 回调。

监听事件

motion.blow 在开启吹气动作的监听时，用户进行了吹气。

事件参数：

- [number] pass - 吹气检测值，该值随着硬件的不同有所不同：在 iOS 平台，该值为 [0,1] 区间范围值，伴随着吹气过程，该值逐渐增大；在 Android 平台，该值始终为 1。

```
document.addEventListener('motion.blow', function(e) {
 alert('检测到吹气');
}, false);


window.WindVane.call('WVMotion', 'listenBlow', {}, function(e) {
 alert('success');
}, function(e) {
 alert('failure:' + JSON.stringify(e));
});
```

WVMotion.stopListenBlow

停止吹气动作的监听。

输入参数

无输入参数。

回调参数

无回调参数，总是进入 success 回调。

```
window.WindVane.call('WVMotion', 'stopListenBlow', {}, function(e) {
 alert('success');
});
```

WVMotion.listenGyro

开启或关闭陀螺仪的监听。

这里的监听陀螺仪， **实际是为了监听手机方向的改变** ，使得 H5 页面可以根据手机方向做出不同反应。在手机中，有两个传感器可以实现这一目的：陀螺仪和重力感应器。陀螺仪可以得到手机空间位置的欧拉角：roll、pitch 和 yaw，重力感应器则可以得到重力在手机三个方向上的分量。

但是，由于在 iOS 和 Android 平台下得到的陀螺仪数据暂时难以统一，而重力感应器的数据更加容易统一，因此 **目前采用重力感应器的数据作为返回值** ，H5 页面可以通过相关算法计算得到手机方向。

输入参数

- [boolean] on - 是开启还是关闭陀螺仪的监听，true 表示开启监听，false 表示关闭监听。
- [number] frequency - 陀螺仪事件的时间间隔（毫秒），会保证两次陀螺仪事件触发的时间间隔大于指定时间。

回调参数

无回调参数，如果成功开启或关闭了陀螺仪的监听，则进入 success 回调，否则进入 failure 回调。

监听事件

motion.gyro 在开启陀螺仪的监听时，手机的方向发生了变化。

事件参数：

- [number] x - 手机 x 方向上的重力分量值，范围是 [-1, 1]。
- [number] y - 手机 y 方向上的重力分量值，范围是 [-1, 1]。
- [number] z - 手机 z 方向上的重力分量值，范围是 [-1, 1]。

```
document.addEventListener('motion.gyro', function(e) {
 alert('检测到手机方向改变');
}, false);


var params = {
 // 是开启还是关闭陀螺仪的监听
 on: true,
 // 陀螺仪事件的时间间隔
 frequency: 100
};
window.WindVane.call('WVMotion', 'listenGyro', params, function(e) {
 alert('success');
}, function(e) {
 alert('failure:' + JSON.stringify(e));
});
```

WVMotion.listeningShake

开启或关闭摇动的监听。

输入参数

- [boolean] on - 是开启还是关闭摇动的监听，true 表示开启监听，false 表示关闭监听。
- [number] frequency - *[可选]*摇动事件的时间间隔（毫秒），会保证两次摇动事件触发的时间间隔大于指定时间，默认为 500。
- [number] shakeThreshold - *[可选]*摇动敏感度，加速度超过敏感度就认为是一次有效摇动，默认为 1.2（VER.WindVane iOS 8.1.0 或更高）。
- [number] shakeNum - *[可选]*摇动次数，仅当摇动了指定次数后才会触发一次摇动事件，默认为 1（VER.WindVane iOS 8.1.0 或更高）。

回调参数

无回调参数，如果成功开启或关闭了摇动的监听，则进入 success 回调，否则进入 failure 回调。

监听事件

motion.shake 在开启摇动的监听时，用户摇动了手机。

事件参数（VER.仅限 WindVane iOS）：

- [number] x - 手机 x 方向上的加速度。
- [number] y - 手机 y 方向上的加速度。
- [number] z - 手机 z 方向上的加速度。

```
document.addEventListener('motion.shake', function(e) {
 alert('检测到摇动');
}, false);


var params = {
 // 是开启还是关闭摇动的监听
 on: true
};
window.WindVane.call('WVMotion', 'listeningShake', params, function(e) {
 alert('success');
}, function(e) {
 alert('failure: ' + JSON.stringify(e));
})
```

WVMotion.vibrate

令手机震动。

输入参数

- [int] duration - *[可选]*震动的持续时间（毫秒），默认由系统自动决定（仅限 WindVane Android，且部分机型该参数无效）。

回调参数

无回调参数，总是进入 success 回调。

```
window.WindVane.call('WVMotion', 'vibrate', {});


var params = {
 // 震动的持续时间
 duration: 7000
};
window.WindVane.call('WVMotion', 'vibrate', params);
```

3.13. 设备环境探测 WVNativeDetector

WVNativeDetector.isSimulator

获取当前环境是否是模拟器

输入参数

无输入参数。

回调参数

回调参数将会在回调方法中传递，如果成功获取模拟器信息，则进入 success 回调，否则进入 failure 回调。

- [boolean] isSimulator - 如果当前环境是模拟器，则为 true，否则为 false。

```
window.WindVane.call('WVNativeDetector', 'isSimulator', {}, function(e) {
 alert('success: ' + JSON.stringify(e));
}, function(e) {
 alert('failure: ' + JSON.stringify(e));
});
```

WVNativeDetector.getCurrentUsage

获取当前的 CPU 和内存使用率信息。

__注意：__调用该接口会存在性能损耗，因此请仅在必要时调用。

输入参数

无输入参数。

回调参数

回调参数将会在回调方法中传递，如果成功获取使用率信息，则进入 success 回调，否则进入 failure 回调。

- [number] cpuUsage - 当前 CPU 使用率。
- [number] memoryUsage - 当前内存使用率，值为 usedMemory / totalMemory。
- [number] usedMemory - 当前手机的内存使用总量，包含当前应用和其它打开的应用的内存使用量，单位：MB。
- [number] totalMemory - 当前手机的内存总量，单位：MB。

```
window.WindVane.call('WVNativeDetector', 'getCurrentUsage', {}, function(e) {
  alert('success: ' + JSON.stringify(e));
}, function(e) {
  alert('failure: ' + JSON.stringify(e));
});
```

WVNativeDetector.getDeviceYear

获取设备的发布日期，可以用于估算设备的性能。iOS 可以基本准确的获得设备的发布日期，Android 则仅是对设备性能的一个大概估算。

一般认为 2012 年之后（含）发布的机器，其性能可以满足绝大多数需要。

输入参数

无输入参数。

回调参数

回调参数将会在回调方法中传递，如果成功获取设备的发布日期，则进入 success 回调，否则进入 failure 回调。

- [number] deviceYear - 设备的发布日期，如 2012。

```
window.WindVane.call('WVNativeDetector', 'getDeviceYear', {}, function(e) {
 alert('success: ' + JSON.stringify(e));
}, function(e) {
 alert('failure: ' + JSON.stringify(e));
});
```

WVNativeDetector.getModelInf

获取设备型号等信息。

输入参数

无输入参数。

回调参数

回调参数将会在回调方法中传递，如果成功获取设备信息，则进入 success 回调，否则进入 failure 回调。

- [string] brand - 设备品牌，例如 "Apple" 或 "Google"。
- [string] model - 设备型号，例如 "iPhone" 或 "iPod touch" 或 "Nexus 5"。
- [string] platform - 设备平台，例如 "iPhone5,2"，仅限 VER.WindVane iOS。
- [string] platformName - 设备平台，例如 "iPhone 5 (CDMA)"，仅限 VER.WindVane iOS。

```
window.WindVane.call('WVNativeDetector', 'getModelInfo', {}, function(e) {
 alert('success: ' + JSON.stringify(e));
}, function(e) {
 alert('failure: ' + JSON.stringify(e));
});
```

WVNativeDetector.getSafeAreaInsets

WindVane iOS

获取当前页面的安全区域信息，仅适用于全屏页面，用于页面对 iOS11/iPhoneX 做兼容。

输入参数

无输入参数。

回调参数

回调参数将会在回调方法中传递，如果成功获取 SafeArea 信息，则进入 success 回调，否则进入 failure 回调。

- [Number] top - SafeArea 的顶部 inset。
- [Number] left - SafeArea 的左边 inset。
- [Number] bottom - SafeArea 的底部 inset。
- [Number] right - SafeArea 的右边 inset。
- [Boolean] cssAvaliable - constant(safe-area-inset-*) 的 CSS 是否可用，等价于判断系统是否是 VER.iOS 11 或更高，且客户端使用 Xcode9 进行编译。

```
window.WindVane.call('WVNativeDetector', 'getSafeAreaInsets', {}, function(e) {
 alert('success: ' + JSON.stringify(e));
}, function(e) {
 alert('failure: ' + JSON.stringify(e));
});
```

3.14. 蜂鸣 WVNotification

WVNotification.beep

令手机蜂鸣。

输入参数

- [int] count - *[可选]*连续蜂鸣的次数，默认为 1。

回调参数

无回调参数，如果成功蜂鸣，则进入 success 回调，否则进入 failure 回调。

```
var params = {
 // 连续蜂鸣的次数
 count: 3
};
window.WindVane.call('WVNotification', 'beep', params, function(e) {
 alert('success');
}, function(e) {
 alert('failure: ' + JSON.stringify(e));
});
```

3.15. 截屏 WVScreen

WVScreen.capture

截取当前 WebView 显示的部分为图片。

__注意：__在 iOS 11 中，要求客户端添加一个新的权限描述 NSPhotoLibraryAddUsageDescription，才允许访问相册。

输入参数

- [String] inAlbum - 是否将截屏结果保存一份到相册中，'true' 表示保存结果到相册，'false' 表示不保存。默认为 'false'。
- [String] type - 截屏的类型，设置为 'view' 可以只截取 WebView 部分，或者 'app' 来截取应用部分，默认为 'view'。设置为 app 后，会包含状态栏背景（无状态栏文本），**一些有软键盘（类似 SmartBar）的 Android 机型上，软键盘部分会留下黑色背景**。WindVane 8.3.0。

回调参数

回调参数将会在回调方法中传递，如果成功截屏，则进入 success 回调，否则进入 failure 回调。

- [string] url - 访问 URL，页面将此 URL 填写给 img 的 src 属性，用于进行截屏结果的预览。 **注意：该 URL 非真实的 CDN URL，浏览器无法访问。**
- [string] localPath - 截屏结果的本地文件路径，该路径可以后续使用 WVCamera.confirmUploadPhoto 上传到 CDN。

```
var params = {
 // 是否将截屏结果保存一份到相册中
 inAlbum: 'true'
};
window.WindVane.call('WVScreen', 'capture', params, function(e) {
 alert('success: ' + JSON.stringify(e));
}, function(e) {
 alert('failure: ' + JSON.stringify(e));
});
```

WVScreen.getOrientation

返回当前 Activity/ViewController 的显示方向（横屏/竖屏）。

输入参数

无输入参数。

回调参数

回调参数将会在回调方法中传递，如果获取显示方向，则进入 success 回调，否则进入 failure 回调。

- [string] orientation - 当前 Activity/ViewController 的显示方向，可能的值为：
  - 'landscape' - 横屏。
  - 'portrait' - 竖屏。
  - 'unknown' - 方向未知。

```
window.WindVane.call('WVScreen', 'getOrientation', {}, function(e) {
 alert('success: ' + JSON.stringify(e));
}, function(e) {
 alert('failure: ' + JSON.stringify(e));
});
```

WVScreen.setOrientation

设置当前 ViewController/Activity 的显示方向（横屏/竖屏）。注意此 JSBridge 会影响到当前 WebView 跳转到的其它页面，所以请在页面跳转时主动恢复显示方向。

输入参数

- [string] orientation - 要设置的 Activity 显示方向，可能的值为：
  - 'default' - 客户端默认显示方向
  - 'landscape' - 横屏，iOS 平台下如果客户端支持，会根据设备方向自动切换左/右横屏
  - 'portrait' - 竖屏
  - 'landscapeRight' - 右横屏 WindVane iOS 8.2.0
  - 'landscapeLeft' - 左横屏 WindVane iOS 8.2.0
  - 'portraitUpsideDown' - 颠倒竖屏 WindVane iOS 8.2.0
  - 'auto' - 根据设备方向自动切换 WindVane iOS 8.2.0

回调参数

无回调参数，如果设置显示方向，则进入 success 回调，否则进入 failure 回调。

```
var params = {
 orientation: 'landscape'
};
window.WindVane.call('WVScreen', 'setOrientation', params, function(e) {
 alert('success: ' + JSON.stringify(e));
```

```
}, function(e) {
 alert('failure: ' + JSON.stringify(e));
});
```

3.16. 向 Native 发送事件 WVStandardEventCenter

WVStandardEventCenter.postNotificationToNative

H5 向 Native 发送事件，具体的事件名称可以是与 Native 业务方商议一致的任意名称。

iOS 事件监听用法

使用标准的消息中心监听事件：

```
[[NSNotificationCenter defaultCenter] addObserver:self selector:@selector(myEventListener)
name:eventName object:nil]

- (void)myEventListener:(NSNotification *)notification {
 // 事件参数可以从 notification.userInfo 中获取。
}
```

Android 事件监听用法

```
@import android.taobao.windvane.service;
@import android.taobao.windvane.standardmodal

// 首先，需要自己实现监听用的 Listener。
public class JsEventListener implements WVEventListener {
 @Override
 public WVEventResult onEvent(int id, WVEventContext ctx, Object... obj) {
  // 所有事件均派发到这里，WVEventId.H5TONATIVE_EVENT 表示 H5 发送过来的事件。
  if (id == WVEventId.H5TONATIVE_EVENT) {
   if (obj[0] instanceof String) {
    String params = (String)obj[0];
```

```
      // 这里的 params 是包含 event 和 param 的 JSON String，需要自行反序列化。
     }
    }
    return null;
   }
  }

  // 然后，注册监听器。
  WVEventService.getInstance().addEventListener(new JsEventListener());
```

输入参数

- [string] event - 要发送的事件名称。
- [object] param - *[可选]*要发送的事件参数。

回调参数

无回调参数，如果成功蜂鸣，则进入 success 回调，否则进入 failure 回调。

```
var params = {
 event: 'eventName',
 param: {
  // 事件要传递的数据。
 }
};
window.WindVane.call('WVStandardEventCenter','postNotificationToNative', params,
function(e) {
 alert('success');
}, function(e) {
 alert('failure: ' + JSON.stringify(e));
});
```

3.17. 信息报告 WVReporter

WVReporter.commitAppMonitorStat

提交 AppMonitor 的性能埋点。

输入参数

- [String] module - 埋点的模块名。
- [String] point - 埋点的监控点。
- [Object] dimensions - 埋点的维度信息，对象的每个值（字符串）都会作为 AppMonitor 性能埋点的维度使用。
- [Object] measures - 埋点的指标信息，对象的每个值（数字）都会作为 AppMonitor 性能埋点的指标使用。

回调参数

无回调参数，如果埋点成功，则进入 success 回调，否则进入 failure 回调。

```
var params = {
 module: 'MyModuleName',
 point: 'MyPoint',
 dimensions: {
  'url': 'test.WindVane.taobao.com',
  'myDim1': '201',
 },
 measures: {
  'value': 10.2,
  'myValue2': 100.2,
 },
};
window.WindVane.call("WVReporter", "commitAppMonitorStat", params);
```

WVReporter.reportDomLoad

报告当前页面的首字节时间和 DomLoad 事件时间，供 H5 性能埋点使用。

WindVane iOS 8.2.0 或更高版本 中，WindVane 会自动捕获 DOMContentLoaded 事件的时间，与页面自己的埋点不会产生冲突。

输入参数

- [number] firstByte - 当前页面的首字节时间（使用 Date.now() 获得的毫秒数）。
- [number] time - 当前页面的 DomLoad 事件时间（使用 Date.now() 获得的毫秒数）。

回调参数

无回调参数，如果报告信息成功，则进入 success 回调，否则进入 failure 回调。

```
(function () {
 var firstByteTime = Date.now();
 document.addEventListener("DOMContentLoaded", function () {
  if (window.WindVane) {
   var params = {
    firstByte: firstByteTime,
    time: Date.now()
   };
   window.WindVane.call("WVReporter", "reportDomLoad", params);
  }
 }, false);
})();
```

WVReporter.reportError

报告当前页面的 JS 错误，会通过 AppMonitor 的 WindVane-JavaScriptError 埋点上传。

WindVane iOS 8.2.0 或更高版本 中，WindVane 会自动捕获 JS 错误，并设置 from = 'WindVane'，便于与业务方自己的埋点区分开来。

输入参数

- [string] file - 发生 JS 错误的 JS 文件。
- [string] url - *[可选]*要上传的 URL，默认使用当前页面的 URL。
- [string] msg - JS 错误信息。
- [string] line - 发生 JS 错误的 JS 行号。
- [string] col - 发生 JS 错误的 JS 列号 WindVane iOS 8.2.0 或更高。
- [string] stack - 发生 JS 错误的 JS 堆栈 WindVane iOS 8.2.0 或更高。
- [string] from - *[可选]*错误埋点的来源，用于区分同样的错误被多次捕获 WindVane iOS 8.2.0 或更高。

回调参数

无回调参数，如果报告信息成功成功，则进入 success 回调，否则进入 failure 回调。

示例

```
(function () {
 if (window.WindVane) {
  var oldOnError = window.onerror
  window.onerror = function (message, file, lineno, colno, error) {
   var params = {
    file: file,
    msg: message
    line: lineno,
    col: colno,
    stack: error ? error.stack : ''
   };
   window.WindVane.call("WVReporter", "reportError", params);
   if (oldOnError) {
    return oldOnError(message, file, lineno)
   }
   return false
  }
 }
})();
```

3.18. 性能统计 WVWebPerformance

WVWebPerformance.timing

返回当前 URL 的资源访问情况。

输入参数

无输入参数。

回调参数

回调参数将会在回调方法中传递，如果获取当前 URL 资源访问情况成功，则进入 success 回调，否则进入 failure 回调。

- [array] resources - 当前 URL 的资源访问列表，每一项包含以下属性：
  - [string] url - 资源的 URL。
  - [string] statusCode - 资源的状态码。
  - [number] start - 资源开始加载时间（相对 pageStart）。
  - [number] end - 资源结束加载时间（相对 pageStart）。

```
window.WindVane.call('WVWebPerformance', 'timing', {}, function(e) {
 alert('success: ' + JSON.stringify(e));
}, function(e) {
 alert('failure: ' + JSON.stringify(e));
});
```

WVWebPerformance.jsBridgeHistory

返回当前 URL 的 JSBridge 访问情况。

输入参数

无输入参数。

回调参数

回调参数将会在回调方法中传递，如果获取当前 URL 的 JSBridge 访问情况成功，则进入 success 回调，否则进入 failure 回调。

- [number] JSBridgeName - 指定名称的 JSBridge 调用次数，JSBridgeName 的格式为 类名.方法名。

```
window.WindVane.call('WVWebPerformance', 'jsBridgeHistory', {}, function(e) {
 alert('success: ' + JSON.stringify(e));
}, function(e) {
 alert('failure: ' + JSON.stringify(e));
});
```

3.19. 应用信息 WVApplication

WVApplication.appState

仅限 WindVane iOS

返回当前应用的状态（是否在前台）。

输入参数

无输入参数。

回调参数

回调参数将会在回调方法中传递，如果成功获取应用状态，则进入 success 回调，否则进入 failure 回调。

- [string] state - 当前应用的状态，可能的值为：
  - 'active' - 应用处于活动状态。
  - 'inactive' - 应用处于前台非活动状态，例如正在下拉通知栏、双击 Home 键的任务管理界面或锁屏等。
  - 'background' - 应用处于后台状态。
  - 'unknown' - 应用状态未知。

```
window.WindVane.call('WVApplication', 'appState', {}, function(e) {
  alert('success: ' + JSON.stringify(e));
}, function(e) {
  alert('failure: ' + JSON.stringify(e));
});
```

WVApplication.getNotificationSettings

仅限 WindVane iOS

返回当前应用的通知状态。

输入参数

无输入参数。

回调参数

回调参数将会在回调方法中传递，如果成功获取应用的通知状态，则进入 success 回调，否则进入 failure 回调。

- [string] status - 当前应用的通知状态，可能的值为：
  - 'authorized' - 用户已允许应用弹出通知。
  - 'denied' - 用户禁止应用弹出通知。
  - 'notDetermined' - 用户尚未决定是否允许应用弹出通知，在 iOS 系统应用尚未请求通知权限前可能返回该值。
  - 'unknown' - 应用通知状态未知。

```
window.WindVane.call('WVApplication', 'getNotificationSettings', {}, function(e) {
 alert('success: ' + JSON.stringify(e));
}, function(e) {
 alert('failure: ' + JSON.stringify(e));
});
```

WVApplication.openSettings

仅限 WindVane iOS

打开当前应用的设置页面。

输入参数

- [string] type - *[可选]*要打开特定类型的设置页面，未传入则直接打开应用设置页。可能的值有：
  - 'Notification' - 打开当前应用的通知设置页面。iOS 系统中仍会显示应用设置页。

回调参数

无回调参数，如果成功打开设置页面，则进入 success 回调，否则进入 failure 回调。

```
window.WindVane.call('WVApplication', 'openSettings', {}, function(e) {
    alert('success: ' + JSON.stringify(e));
}, function(e) {
    alert('failure: ' + JSON.stringify(e));
});
```

3.20. 图片保存 WVImage

WVImage.saveImage

WindVane Android 8.6.1.3-U4-3 或者更高 手淘 Android 9.5.0 或者更高

允许保存 Http（s）地址或者 base64 个市的图片到相册中

类型	名称
String	url 图片 https 地址或者 base64 图片

回调参数

毁掉参数将会在回掉方法中传递，如果成功保存，则进入 secces 回调；如果保存失败，则进入 failure 回调。

示例：

```
Var param = {
        url: 'https://gw.alicdn.com/tfscom/TB1tU_hqKL2gK0jSZFmXXc7iXXa.png',//需要保存的图片链接
};
Window.WindVane.call ( 'WVImage','saveImage',params,funcation(){
        Alert('saveImage success')
},function(e){
        Alert('saveImage failure:' + JSON.stringify(e));
});
```

3.21. 预加载包 WVPackageAppInfo

WVPackageAppInfo.localPathForURL

返回指定 URL 的本地资源路径，资源可能不存在（如不属于预加载包，或者预加载包未安装）。

输入参数

- [string] url - 要获取本地资源路径的 URL。

回调参数

回调参数将会在回调方法中传递，如果获取本地资源路径成功，则进入 success 回调，否则进入 failure 回调。

- [string] localPath - URL 对应本地资源的路径。

```
var params = {
 url: 'http://h5.m.taobao.com/app/tjb/www/index3.html'
}
window.WindVane.call('WVPackageAppInfo', 'localPathForURL', params, function(e) {
 alert('success: ' + JSON.stringify(e));
}, function(e) {
 alert('failure: ' + JSON.stringify(e));
});
```

3.22. 其它特性 Other Features

页面加载完毕事件 WindVaneReady

当页面加载完毕后，WindVane 会引发 **WindVaneReady** 事件，页面可以监听这一事件进行一些操作。页面刷新、页面历史记录回退同样都会触发这个事件。

```
document.addEventListener('WindVaneReady', function(e) {
 alert('WindVaneReady');
}, false);
```

应用切换后台事件 Background

当应用被切换到后台（如切换应用、打开通知栏等操作）时，WindVane 会引发 **WV.Event.APP.Background** 事件，页面可以监听这一事件进行一些操作。

由于 Android 平台的限制，即使从当前 WebView 切换到其它 Activity，也会引发 **WV.Event.APP.Background** 事件，而不仅仅是切换到后台时。

__注意：__应用被切换到后台时能做的操作较少，可能会被延迟到页面重新激活时才执行。而且， **务必不要** 在该事件中弹 alert，会导致一些 Android 机型出现严重 BUG，打开任何页面都是空白。

WindVane iOS VER.5.6.0 或更高，更新了 Background 事件的触发条件，保持与 Android 的一致：从当前 WebView 切换到其它 Native 或 WebView 时也会触发 Background 事件。

同时增加了一个事件参数 **to**，用于区分从当前 WebView 切换到了何处，其可能的值为：

- 'background' - 表示从当前 WebView 切换到了后台。

如果 to 参数不存在，表示无法确定从当前 WebView 切换到了何处。

```
document.addEventListener('WV.Event.APP.Background', function(e) {
 // 一些机型退出时弹 alert 会出问题。
 console.log('Event Background');
}, false);
```

应用激活事件 Active

当应用被切换到后台再切换回来时，WindVane 会引发 **WV.Event.APP.Active** 事件，页面可以监听这一事件进行一些操作。

由于 Android 平台的限制，即使从其它 Activity 切换到当前 WebView，也会引发 **WV.Event.APP.Active** 事件，而不仅仅是从后台切换到前台时。

WindVane iOS VER.5.6.0 或更高，更新了 Active 事件的触发条件，保持与 Android 的一致：从其它 Native 或 WebView 切换到当前 WebView 时也会触发 Active 事件。

同时增加了一个事件参数 **from**，用于区分从何处切换到当前 WebView，其可能的值为：

- 'background' - 表示从后台切换到了当前 WebView。
- 'webview' - 表示从其它 WebView 切换到了当前 WebView。

如果 from 参数不存在，表示无法确定从何处切换到当前 WebView。

VER.WindVane 6.3.0 或更高，添加了一个 **data** 参数，用来获取从 WebAppInterface.pop 中传递的数据。

```
document.addEventListener('WV.Event.APP.Active', function(e) {
 alert('Event Active');
}, false);
```

下拉刷新 Pull Refresh

仅限 WindVane iOS

WindVane iOS 在默认情况下，会自动启用下拉刷新。要禁止下拉刷新，请将以下元数据添加到 HTML 的 HEAD 中：

```
<meta id="stopUsePullRefresh" value="true">
```

在下拉刷新的提示部分，可以选择是否显示当前页面的域名。要显示当前域名，请将以下元数据添加到 HTML 的 HEAD 中：

```
<meta id="urlInLaw" value="true">
```

无限滚动 Infinite Scroll

仅限 WindVane iOS

当页面滚动到最下方，可以通过无限滚动来动态的加载新内容。要启用无限滚动，请将以下元数据添加到 HTML 的 HEAD 中：

```
<meta id="infiniteScroll" value="true">
```

在 WindVane iOS 中，请将以下元数据也添加到 HEAD 中：

```
<meta id="urlInLaw" value="true">
```

urlInLaw 的 value 请根据需要设置，这里必须添加 urlInLaw，是由于 WindVane iOS 遗留的 BUG。

添加了元数据后，就可以收到无限滚动的事件 **WV_INFINIT_SCROLL**，在该事件中可以动态的获取新内容，并添加到页面的末尾。新内容获取到之后，需要调用 **WVScrollWidget.hiddenInfinitScroll** 这一 API 隐藏无限滚动时的加载提示。

```
document.addEventListener('WV_INFINIT_SCROLL', function(e) {
 var data = 'New document element for infinit scroll';
 window.WindVane.call('WVScrollWidget', 'hiddenInfinitScroll', {}, function(e) {
  var newDiv = document.createElement('div');
  newDiv.innerHTML = data;
  document.body.appendChild(newDiv);
 });
}, false);
```

4. 前端框架兼容性

我们的 EMAS 跨平台产品 H5 方案能够兼容多种前端框架编码的代码项目，只要是基于 webpack、gulp、grunt 打包，都可以支持。采用业界常用的几种前端框架开发的项目，可以参考以下文档来了解相关内容。

Vue.js 工程

基于 Vue.js 编码的项目工程，无需任何代码改动，即可直接使用 EMAS 跨平台产品。例如采用 VuxUI 库开发的代码：

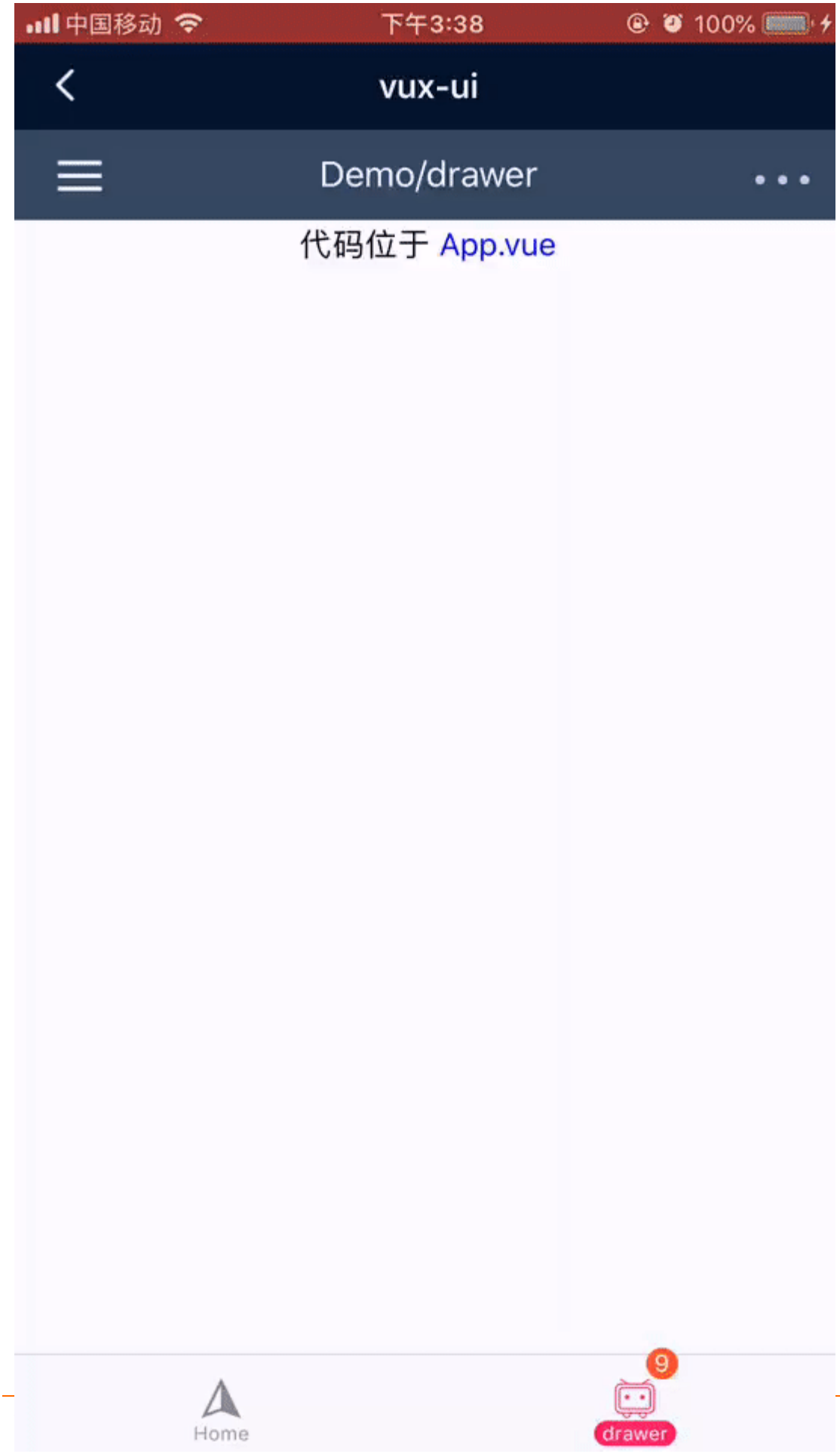
```
<template>
  <div>
    <drawer
      width="200px;"
      :show.sync="drawerVisibility">


    <div slot="drawer">
      <!-- 菜单内容 -->
    </div>
```

```
    <!-- rourer-view 作为默认插槽内容 -->
    <div>
      <router-view></router-view>
    </div>

    </drawer>
  </div>
</template>
```

使用「EMAS 演示」App 扫下面的二维码，可体验效果。


中国移动
下午3:38
100%
vux-ui
Demo/drawer
代码位于 App.vue
9
Home
drawer

React 工程

基于 React 编码的项目工程，无需任何代码改动，即可直接使用 EMAS 跨平台产品。例如采用 Ant Design MobileUI 库开发的代码：

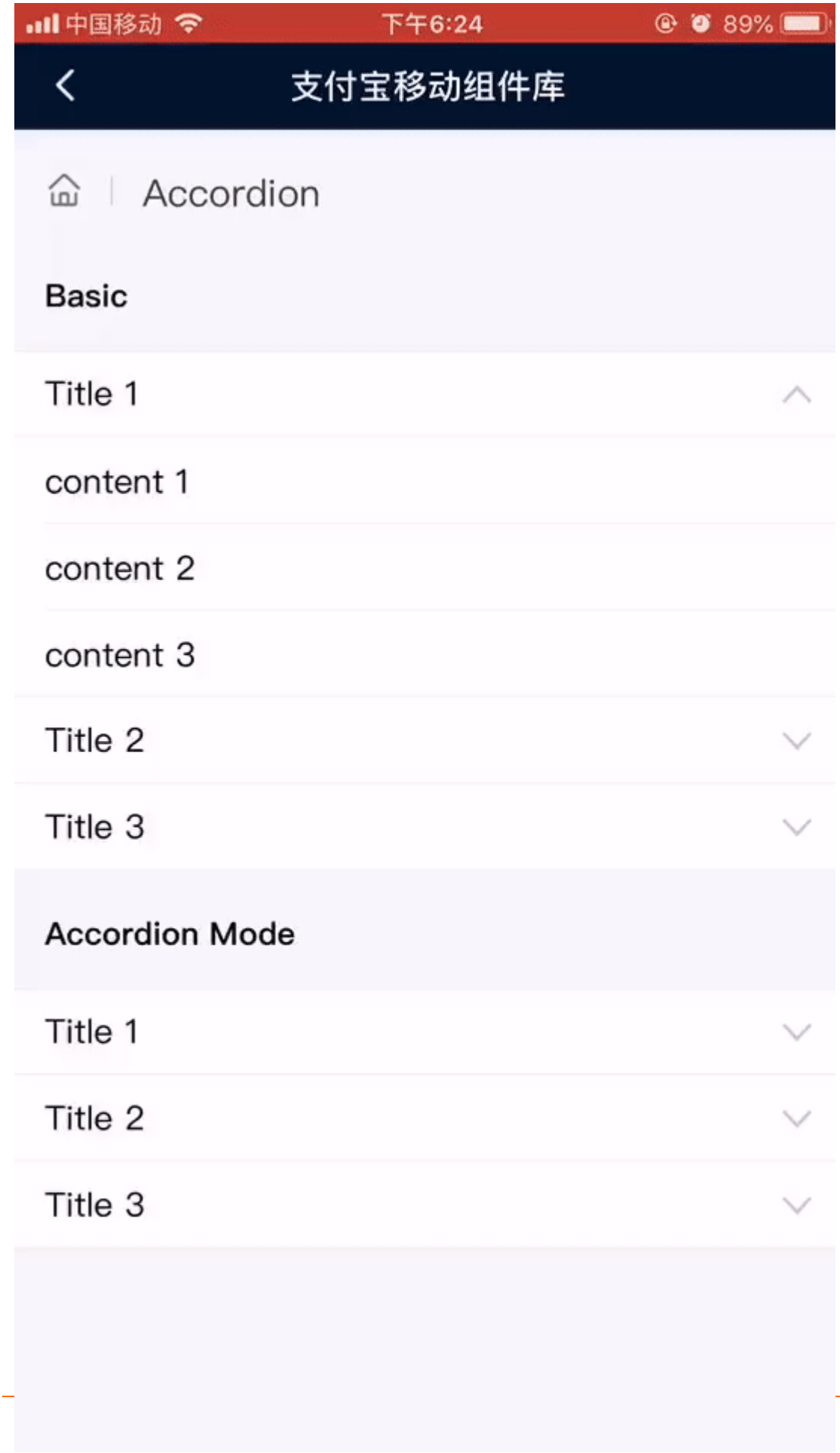
```
import { Accordion, List } from 'antd-mobile';


class AccordionExmple extends React.Component {
  onChange = (key) => {
    console.log(key);
  }
  render() {
    return (
      <div style={{ marginTop: 10, marginBottom: 10 }}>
        <Accordion defaultActiveKey="0" className="my-accordion" onChange={this.onChange}>
          <Accordion.Panel header="Title 1">
            <List className="my-list">
              <List.Item>content 1</List.Item>
              <List.Item>content 2</List.Item>
              <List.Item>content 3</List.Item>
            </List>
          </Accordion.Panel>
          <Accordion.Panel header="Title 2" className="pad">this is panel content2 or
other</Accordion.Panel>
          <Accordion.Panel header="Title 3" className="pad">
            text text text text text text text text text text text text text text text
          </Accordion.Panel>
        </Accordion>
      </div>
    );
  }
```

```
}

ReactDOM.render(<AccordionExmple />, mountNode);
```

使用「EMAS 演示」App 扫下面的二维码，可体验效果。


中国移动
下午6:24
89%
支付宝移动组件库
Accordion
Basic
Title 1
content 1
content 2
content 3
Title 2
Title 3
Accordion Mode
Title 1
Title 2
Title 3

Angular 工程

基于 Angular 编码的项目工程，无需任何代码改动，即可直接使用 EMAS 跨平台产品。例如采用 NG-ZORRO-MOBILEUI 库开发的代码：

```
import { Component } from '@angular/core';


@Component({
  selector: 'demo-stepper-basic',
  template: `
    <List>
      <ListItem [extra]="stepperNgModel">Show number value</ListItem>
      <ListItem [extra]="stepper">Show number value</ListItem>
      <ListItem [extra]="stepperDisabled">Disabled</ListItem>
    </List>
    <ng-template #stepper>
      <Stepper [value]="value" [min]="1" [max]="3" [showNumber]="true"
(onChange)="change($event)"></Stepper>
    </ng-template>
    <ng-template #stepperDisabled>
      <Stepper [defaultValue]="6" [min]="1" [max]="10" [disabled]="true"
[showNumber]="true"></Stepper>
    </ng-template>
    <ng-template #stepperNgModel>
      <Stepper [(ngModel)]="value1" [min]="1" [max]="10" [showNumber]="true"
(ngModelChange)="change($event)"></Stepper>
    </ng-template>
  `,
  styles: [``]
})
export class DemoStepperBasicComponent {
```

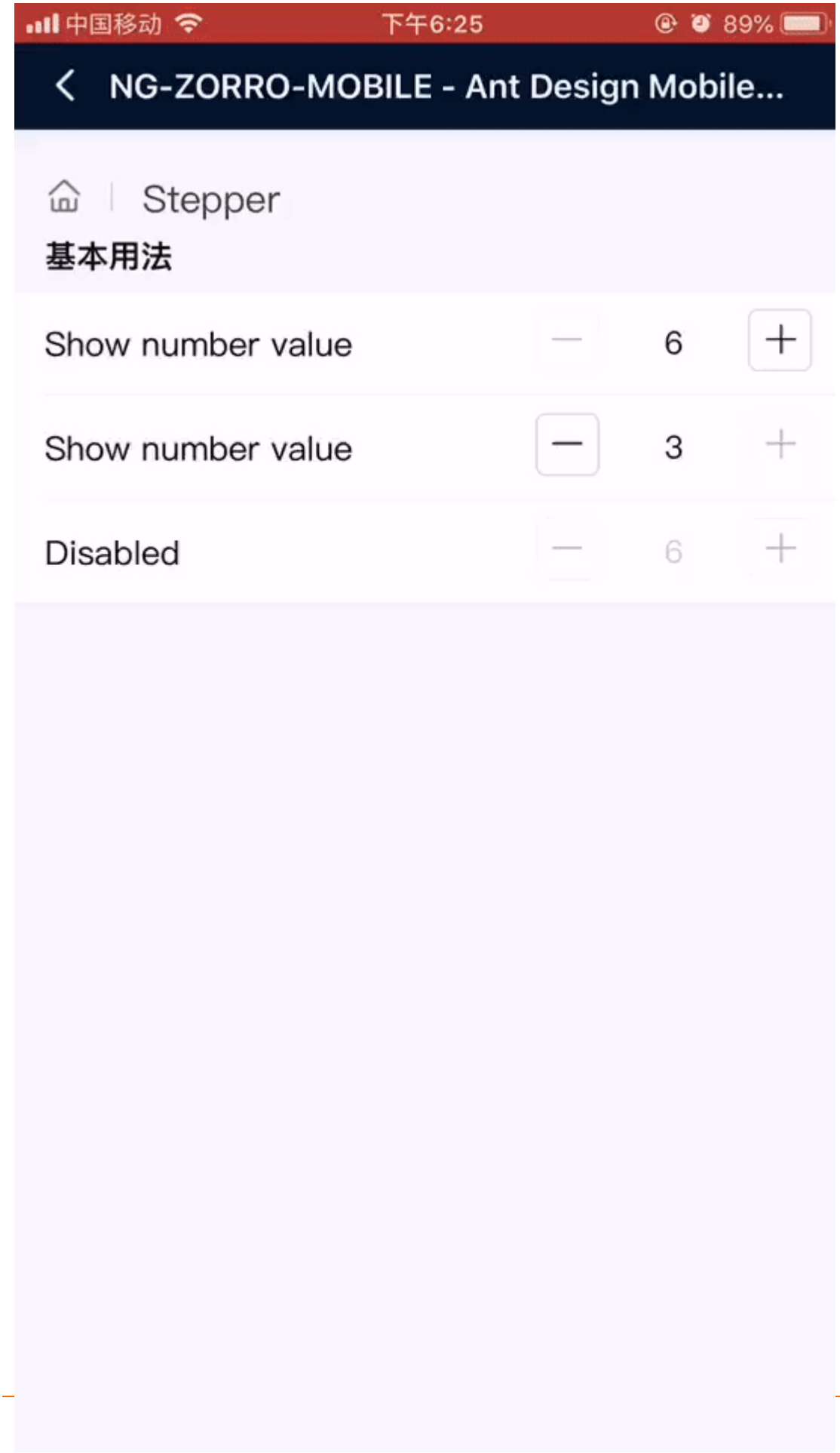
```
    value = 3;
    value1 = 6;

    constructor() {}

    change($event) {
      console.log($event, 'change');
    }

  }
```

使用「EMAS 演示」App 扫下面的二维码，可体验效果。


中国移动
下午6:25
89%
NG-ZORRO-MOBILE - Ant Design Mobile...
Stepper
基本用法
Show number value
6
Show number value
3
Disabled
6

5. 从其他框架迁移

业界还存在一些其他的跨平台技术框架，如果你是采用这些技术框架实现的移动 App，可以阅读以下内容来了解如何从这些技术框架迁移到 EMAS 跨平台 H5 方案。

从 AppCan 迁移

AppCan 开发模式中，界面布局等基本要素都是使用标准 HTML 技术进行开发的，通过调用其提供的 JSSDK 和插件 API 来实现业务需求的开发。

HTML 和 css 代码

这些和界面相关的代码不需要任何修改就可以用 EMAS 跨平台提供的 H5 容器（WindVane）来渲染。

JSSDK 的调用

AppCan 已经封装好的一些 js 库，如 appcan.jsappcan.listview.js 是标准的 js 库。调用到这些库的代码也不需要任何修改就可以用 EMAS 跨平台提供的 H5 容器（WindVane）中正常执行。

插件 API 的调用

这里所谓的插件 API 其实就是 Native 端通过 jsbridge 暴露一些端上能力供前端页面调用。这一部分功能是和 Native 端的 H5 容器强耦合的，所以无法兼容，需要用户在调用类似 uexCamerauexLocation 的地方替换成 EMAS 跨平台提供的 JS API 的调用。我们大致梳理了一下 AppCan 中，通过 JS API 或者其他方法可以替代的几个插件。如果调用了其他插件，可以通过 Native 端使用 JSBridge 扩展 JS API 的办法来补齐这些功能。

uexApplePay

该插件实现了 iPhone 手机的 Apple Pay 支付功能。

可以使用支付宝支付

uexCall(官方已废弃)

打电话功能插件

可以使用标准 HTML 标签来实现

```
<a href="tel:400-0000-688">400-0000-688</a>
```

uexCamera

调用设备照相机拍摄照片,成功后返回相关图片存储地址.

可以使用 JS API 的 WVCamera.takePhoto

uexClipboard

系统剪贴板插件

可以使用 JS API 的 Base.copyToClipboard

uexContact

系统联系人插件

可以使用 JS API 的 WVContacts.choose

uexControl

日期选择器插件

可以使用 pickadate.js

uexDevice

设备信息插件

可以使用 JS API 的 WVNativeDetector.getModelInf

uexLocation

定位插件

可以使用 JS API 的 WVLocation.getLocation

uexSensor

设备传感器插件,设备传感器.

可以使用 JS API 的 WVMotion.listenGyro

uexPDFReader

PDF 阅读器插件

可以使用 pdf.js

uexVideo

视频播放插件

可以使用标准 HTML 标签来实现

```
<video width="320" height="240" controls>
    <source src="movie.mp4" type="video/mp4">
    <source src="movie.ogg" type="video/ogg">
</video>
```

uexDownloaderMgr

文件下载插件

可以使用以下代码来实现

```
window.open(url)
```

uexWebSocket

webSocket 插件

可以使用 websocket

从 Cordova 迁移

Cordova 的开发模式和 AppCan 非常类似，界面布局等基本要素都是使用标准 HTML 技术进行开发的，通过调用插件 API 来实现业务需求的开发。

HTML 和 css 代码

这些和界面相关的代码不需要任何修改就可以用 EMAS 跨平台提供的 H5 容器（WindVane）来渲染。

JSSDK 的调用

Cordova 已经封装好的一些 js 库，如 cordova.js 是标准的 js 库。调用到这些库的代码也不需要任何修改就可以用 EMAS 跨平台提供的 H5 容器（WindVane）中正常执行。

插件的调用

这里所谓的插件其实也是 Native 端通过 jsbridge 暴露一些端上能力供前端页面调用。这一部分功能是和 Native 端的 H5 容器强耦合的，所以也无法兼容，需要用户在调用类似 cordova-plugin-cameracordova-plugin-geolocation 的地方替换成 EMAS 跨平台提供的 JS API 的调用。我们大致梳理了一下 Cordova 中，通过 JS API 或者其他方法可以替代的几个插件。如果调用了其他插件，可以通过 Native 端使用 JSBridge 扩展 JS API 的办法来补齐这些功能。

cordova-plugin-contacts(联系人)

可以使用 JS API 的 WVContacts

cordova-plugin-device(设备)

可以使用 JS API 的 WVNativeDetector

cordova-plugin-geolocation(地理位置)

可以使用 JS API 的 WVLocation

cordova-plugin-network-information(网络信息)

可以使用 JS API 的 WVNativeDetector

cordova-plugin-media-capture(媒体捕获)

可以使用 JS API 的 WVAudio 音频，WVScreen 截图

cordova-plugin-camera(相机)

可以使用 JS API 的 WVCamera

cordova-plugin-dialogs(对话框)

可以使用 JS API 的 Widget

cordova-plugin-inappbrowser(InAppBrowser 打开 Web 浏览器)

可以通过 window.open(url)打开链接，或者通过 JS API 的 Base.openBrowser

cordova-plugin-vibration(震动)

可以使用 JS API 的 WVNotification

建议

AppCan 或者 Cordova 的编码模式，还是停留在比较原始的阶段，落后于业界流行的各种前端框架的编码思想。建议客户使用 Vue.js、React 等主流的前端框架，配合 EMAS 跨平台 H5 容器的能力，打造先进、稳定的跨平台 Hybrid 应用。